PRIVACY-PRESERVING AUTHENTICATION AND AUTHORIZATION FOR VANETS USING BLOCKCHAIN AND ZERO-KNOWLEDGE PROOFS

A Project Report Submitted

in Partial Fulfilment of the Requirements

for the Degree of

Bachelor of Technology

in

Computer Science and Engineering

with Specialization in

Cyber Security

by

Ben Savio - 2023BCY0009

Harsh Kumar Gour - 2023BCY0010

Byreddy Varshith Reddy - 2023BCY0063

Andela Harsha Vardhan - 2023BCY0024



to

DEPARTMENT OF CSE-CYBER SECURITY

INDIAN INSTITUTE OF INFORMATION TECHNOLOGY

KOTTAYAM-686635, INDIA

November 2026

DECLARATION

We, **Ben Savio** (**Roll No: 2023BCY0009**), **Harsh Kumar Gour** (**Roll No: 2023BCY0010**), **Byreddy Varshith Reddy** (**Roll No: 2023BCY0063**) and **Andela Harsha Vardhan** (**Roll No: 2023BCY0024**), hereby declare that this report entitled **"Privacy-Preserving Authentication and Authorization for VANETs Using Blockchain and Zero-Knowledge Proofs"** submitted to Indian Institute of Information Technology Kottayam towards partial requirement of **Bachelor of Technology** in **Computer Science and Engineering** with specialization in **Cyber Security** is an original work carried out by us under the supervision of **Dr. Deepak Jose** and has not formed the basis for the award of any degree or diploma, in this or any other institution or university. We have sincerely tried to uphold the academic ethics and honesty. Whenever an external information or statement or result is used then, that have been duly acknowledged and cited.

Kottayam-686635
November 2026

Ben Savio
Harsh Kumar Gour
Byreddy Varshith Reddy
Andela Harsha Vardhan

CERTIFICATE

This is to certify that the work contained in this project report entitled **"Privacy-Preserving Authentication and Authorization for VANETs Using Blockchain and Zero-Knowledge Proofs"** submitted by **Ben Savio** (**Roll No: 2023BCY0009**), **Harsh Kumar Gour** (**Roll No: 2023BCY0010**), **Byreddy Varshith Reddy** (**Roll No: 2023BCY0063**) and **Andela Harsha Vardhan** (**Roll No: 2023BCY0024**) to the Indian Institute of Information Technology Kottayam towards partial requirement of **Bachelor of Technology** in **Computer Science and Engineering** with specialization in **Cyber Security** has been carried out by them under my supervision and that it has not been submitted elsewhere for the award of any degree.

Kottayam-686635 (Dr. Deepak Jose)

November 2026 Project Supervisor

ACKNOWLEDGEMENT

We express our sincere gratitude to our project guide, Dr. Deepak Jose, for his guidance, encouragement and valuable suggestions throughout this work; his feedback at the Phase I review shaped the direction of the implementation and its evaluation. We thank the Department of CSE-Cyber Security, Indian Institute of Information Technology Kottayam, for the facilities and the academic environment that made this project possible, and the faculty and staff of the department for their support. We also acknowledge the authors and maintainers of the open tools and datasets this work builds on: ns-3, SUMO, the Luxembourg SUMO Traffic (LuST) scenario, the VeReMi Extension dataset and Hyperledger Fabric.

Ben Savio Harsh Kumar Gour Byreddy Varshith Reddy Andela Harsha Vardhan

ABSTRACT

Vehicles in a vehicular ad hoc network (VANET) broadcast safety messages about ten times a second. These messages must be authentic, must not let an observer track a vehicle, and must stay small enough for a shared radio channel. Existing schemes trade these goals against each other.

This project designs, implements and evaluates a scheme that combines them. A trusted authority registers each vehicle's pseudonyms in a Merkle tree and publishes only its root on a permissioned Hyperledger Fabric ledger. A vehicle authenticates to a roadside unit (RSU) with a Schnorr zero-knowledge proof of membership bound to the RSU's fresh nonce, the requested service and its vehicle class, and receives a short-lived ticket that it uses to sign its safety messages and to hand over between RSUs. Smart-contract policies decide which vehicle classes may use which service. RSUs run a misbehaviour detector, report suspicious pseudonyms to the ledger, where two reports revoke a pseudonym, and a threshold of RSU organizations can jointly unmask a revoked vehicle, which then loses all of its pseudonyms. The work further adds TESLA broadcast authentication, implicit certificates, dead-reckoning message triggering, revocation filters and emergency vehicles.

The system runs on ns-3 (IEEE 802.11p) with SUMO mobility on a grid and on real Luxembourg traffic (LuST), with measured cryptographic costs, against an IEEE 1609.2-style certificate baseline and a central authority. At 150 vehicles on the grid, a zero-knowledge login takes 1.59 ms, as fast as presenting a certificate (1.75 ms) and far faster than a central authority (22.49 ms). The scheme puts 3.9 times less safety traffic on the channel than certificate-signed messages, lowers the channel busy ratio from 0.147 to 0.047, and lets vehicles reject messages from revoked senders (0.001 still accepted), while every attacker is caught and no honest vehicle is revoked. Its login costs about as much computation as BPA and far less than ZAMA and B-TSCA, priced with the same unit costs. On the VeReMi Extension dataset the detector reaches an F1 score of 0.963, up from 0.718 for plausibility rules alone. A Fabric 3.1.5 benchmark compares Raft and SmartBFT ordering for the ledger operations.

Keywords: VANET, zero-knowledge proof, permissioned blockchain, Hyperledger Fabric, conditional privacy, revocation, misbehaviour detection, ns-3, SUMO.

Contents

Acknowledgement **iv**

Abstract **v**

List of Figures **ix**

List of Tables **ix**

List of Abbreviations **xi**

1 Introduction **1**
1.1 Vehicular Ad Hoc Networks 1
1.2 Motivation 2
1.3 Objectives 2
1.4 Contributions 3
1.5 Team and Division of Work 3
1.6 Report Structure 4

2 Background and Problem Statement **5**
2.1 Security Requirements 5
2.2 Threat Model 6
2.3 Cryptographic Building Blocks 6
2.3.1 Schnorr Proof of Knowledge 6
2.3.2 Merkle Trees 7
2.3.3 Threshold Identity Escrow 7
2.3.4 Signatures, TESLA and Implicit Certificates 8
2.3.5 Bloom Filters 8
2.4 Permissioned Blockchain: Hyperledger Fabric 8
2.5 Misbehaviour Detection 9
2.6 Problem Statement 10

3 Literature Review **11**
3.1 PKI and Pseudonymous Certificates 11
3.2 ECC and Certificateless Authentication 11
3.3 Hardware and Game-Theoretic Approaches 12
3.4 Blockchain-Based Authentication 12
3.5 Zero-Knowledge Authentication 12
3.6 Blockchain with Zero-Knowledge Proofs 13
3.7 Misbehaviour Detection and Location Privacy 13
3.8 Comparison and Research Gap 13

4 Proposed Scheme **15**
4.1 Architecture and Design Principle . . . 15
4.2 Registration . . . 16
4.3 Authentication . . . 16
4.4 Handover and Pseudonym Change . . . 18
4.5 Safety Messages . . . 18
4.6 Misbehaviour Detection and Reporting . . . 18
4.7 Revocation and Threshold Identity Reveal . . . 19
4.8 Ledger Interface . . . 19
4.9 Efficiency and Robustness Mechanisms . . . 20
4.9.1 Smaller Messages: Digests, Compact Encoding, Implicit Certificates . . . 21
4.9.2 Cheaper Authentication: TESLA with Adaptive Signing . . . 21
4.9.3 Fewer Messages: Dead Reckoning . . . 21
4.9.4 Fewer Handovers: RSU Clusters and Ticket Renewal . . . 22
4.9.5 Load-Triggered Batch Verification . . . 22
4.9.6 Revocation that Reaches Vehicles . . . 22
4.9.7 Trust-Weighted Reports . . . 22
4.9.8 Signal-Strength Check . . . 23
4.10 Emergency Vehicles . . . 23
4.11 Security Analysis . . . 24

5 Implementation **26**
5.1 Overview . . . 26
5.2 Mobility . . . 27
5.3 Radio Network . . . 27
5.4 Protocol . . . 28
5.5 Ledger . . . 29
5.6 Detection . . . 29
5.7 Evaluation Framework . . . 30
5.8 Live Demonstration Tools . . . 30
5.9 Testing . . . 31

6 Results and Discussion **32**
6.1 Experimental Setup . . . 32
6.2 Comparison on the Grid . . . 33
6.3 Real Traffic: LuST . . . 34
6.4 Misbehaviour Detection . . . 37
6.5 Compromised RSUs . . . 39
6.6 Blockchain Performance . . . 39
6.7 Sensitivity to the Vehicle Processor . . . 40
6.8 Emergency Vehicles . . . 42
6.9 Comparison with Published Schemes . . . 42
6.10 Discussion . . . 43

7 Conclusion and Future Work **45**
7.1 Summary 45
7.2 Limitations 46
7.3 Future Work 46
7.4 Closing Remarks 47

A Reproducing the Results **48**

Bibliography **49**

List of Figures

4.1 Entities and the information each exchanges. The blockchain holds only state that several RSUs must share and that must be auditable; it is not on the path of individual safety messages. 15
4.2 Authentication, ticket issue and the first safety message 17

5.1 Simulation and evaluation pipeline 26
5.2 Live map of a running ns-3 simulation (grid, 50 vehicles, with emergency vehicles). The selected emergency vehicle shows its registered class and its `emergency-priority` ticket; revoked attackers are marked with a cross; the feed narrates detection and revocation. 31

6.1 Grid scenario over vehicle density (mean and 95 % CI over 5 seeds) . 35
6.2 LuST traffic at three times of day (mean and 95 % CI over 3 seeds) . 36
6.3 Emergency vehicles on LuST: ticket times and warning coverage . . . 41

List of Tables

3.1 Families of VANET authentication schemes (adapted from the Phase I review) 13

4.1 Chaincode functions and who may call them 19
4.2 Mechanisms added to the core scheme 20

5.1 Code organisation 27
5.2 IEEE 802.11p parameters in ns-3 28
5.3 Measured cost of each cryptographic operation on the development PC (ms) 29

6.1 Common parameters of the network experiments 32
6.2 Grid scenario, 150 vehicles: latency and overhead (ns-3, mean ± 95 % CI over 5 seeds) 33
6.3 Grid scenario, 150 vehicles: security and privacy 33
6.4 Welch's t-test of this work against each baseline (p-values; bold: $p < 0.05$) 34
6.5 Grid scenario at 50, 100 and 150 vehicles 35
6.6 LuST (Luxembourg) traffic: low (06:00), mid (14:00) and peak (08:00) windows 36
6.7 Misbehaviour detection on VeReMi Extension (5 vehicle-disjoint splits) 37
6.8 Detection per attack type 38
6.9 Share of attackers 38

6.10 Compromised RSUs: 25 % of the RSUs report random honest vehicles (100 vehicles, 10 seeds) 39
6.11 Hyperledger Fabric 3.1.5: commit latency of RecordAuth under load . 40
6.12 Vehicle processor speed: measured times × 1, 5, 10 40
6.13 Emergency vehicles (2 % of the honest vehicles) on LuST traffic: ticket times 41
6.14 Emergency vehicles on LuST traffic: warnings and the fake-emergency attack 41
6.15 Cost of one authentication and one re-authentication (handover), priced with BPA's unit costs (point multiplication 0.233 ms, hash 0.0055 ms, pairing 4.756 ms) 43

List of Abbreviations

BBS	Boneh–Boyen–Shacham signatures (anonymous credentials)
BFT	Byzantine fault tolerance
BSM	Basic Safety Message (“here I am” broadcast, SAE J2735)
CBR	Channel busy ratio
CI	Confidence interval
DR	Dead reckoning
ECDSA	Elliptic Curve Digital Signature Algorithm
ECQV	Elliptic Curve Qu–Vanstone implicit certificates
F1	Harmonic mean of precision and recall
ITS	Intelligent Transportation Systems
LuST	Luxembourg SUMO Traffic scenario
MSP	Membership Service Provider (Hyperledger Fabric)
OBU	On-board unit
PDR	Packet delivery ratio
PKI	Public-key infrastructure
RSU	Roadside unit
SCMS	Security Credential Management System
SUMO	Simulation of Urban MObility
TA	Trusted authority
TESLA	Timed Efficient Stream Loss-tolerant Authentication
V2I / V2V	Vehicle-to-infrastructure / vehicle-to-vehicle
VANET	Vehicular ad hoc network
ZKP	Zero-knowledge proof

Chapter 1

Introduction

Road vehicles are becoming networked computers. Through dedicated short-range radio, a vehicle can tell the vehicles around it where it is, how fast it is going and that it is braking, long before a driver could see the danger. Intelligent Transportation Systems (ITS) build collision warnings, cooperative intersection management and emergency-vehicle preemption on these messages. Their value depends on one question that the radio alone cannot answer: can the message be trusted? This report describes a system that answers it without giving up the privacy of the people in the vehicles, and measures what that costs on a realistic network.

1.1 Vehicular Ad Hoc Networks

A vehicular ad hoc network (VANET) consists of three kinds of participants:

- **On-board units (OBUs):** the radio computers in vehicles. They broadcast a *Basic Safety Message* (BSM) [1], also called a cooperative awareness message in Europe [2], about ten times per second, with position, speed and heading.
- **Roadside units (RSUs):** fixed radios at junctions with a wired backbone connection. They authenticate vehicles, relay information and act as the network's local infrastructure.
- **A trusted authority (TA):** for example a transport department, which registers vehicles. It is not on the communication path during normal operation.

Vehicles talk to each other (V2V) and to RSUs (V2I) over IEEE 802.11p [3, 4], a Wi-Fi variant for the 5.9 GHz band that works without association, so a vehicle can broadcast to whoever is in range. Links last seconds, topology changes constantly, and every vehicle in range shares one channel.

1.2 Motivation

Three requirements pull in different directions:

1. **Trust.** A forged BSM reporting a phantom vehicle can trigger an emergency brake; a forged emergency-vehicle message can clear an intersection for an attacker. Receivers must verify that every message comes from a registered vehicle, and misbehaving vehicles must be removed.

2. **Privacy.** A vehicle that broadcasts its position ten times a second under a fixed identifier can be tracked through a city by anyone with a receiver [5]. Authentication must therefore not reveal *which* vehicle is speaking, yet an authorized investigation must still be able to identify a vehicle that attacked the network (conditional privacy).

3. **Efficiency.** Security bytes are paid ten times a second by every vehicle on a shared channel. Above roughly 60 % channel occupancy, broadcasts start to collide and safety messages are lost. Verification must also finish within milliseconds on modest automotive hardware.

Certificate-based schemes [6, 7] solve the first by keeping a central authority on the trust path and the second by changing pseudonym certificates, at the cost of large certificate pools and revocation lists. Zero-knowledge schemes [8, 9] hide the credential but lack a shared, auditable record of authorization and revocation across RSUs. Blockchain-based schemes [10, 11, 12] provide that shared state but add consensus latency. The Phase I review of this project identified the open question:

> *Can privacy-preserving ZKP authentication be combined with permissioned blockchain authorization and lifecycle management without unacceptable latency, computation or communication overhead?*

1.3 Objectives

1. Design a scheme in which a vehicle proves registration with a zero-knowledge proof, an RSU decides access with a ledger policy, and registration roots, authorizations, misbehaviour reports, revocations and identity-reveal approvals live on a permissioned blockchain.

2. Support the whole lifecycle: registration, authentication, handover, renewal, signed safety messages, detection and reporting, revocation and threshold identity reveal.
3. Implement it completely: protocol, chaincode and gateway on Hyperledger Fabric, detector, and a network simulation on ns-3 with SUMO mobility.
4. Measure it on realistic traffic against established baselines, report means with confidence intervals and significance tests, and state the limitations.
5. Reduce the measured latency and overhead without weakening the security model, and handle emergency vehicles as a privileged class.

1.4 Contributions

- A complete authentication and authorization lifecycle that keeps the blockchain off the critical path of safety messages: proofs are verified at the RSU, and the ledger holds only the state that has to be shared and audited (Chapter 4).
- Mechanisms that make the scheme cheap on the radio channel and robust against misbehaving vehicles and RSUs: TESLA, implicit certificates, RSU clusters, dead reckoning, revocation filters, fast quarantine, trust-weighted reports and batch verification (Section 4.9).
- An emergency-vehicle extension: class-bound credentials, a policy-restricted service, signed siren alerts that cannot be forged under a normal ticket, and a priority lane at RSUs.
- A reproducible evaluation framework: ns-3 802.11p with real Luxembourg traffic, measured cryptographic costs on a per-node processor model, the VeReMi Extension detection benchmark, a Fabric consensus benchmark, ten parameter sweeps, and a live browser visualisation for demonstrations (Chapters 5 and 6).

1.5 Team and Division of Work

The implementation was divided into four parts with written interfaces between them (`docs/interfaces.md` in the repository). **Ben Savio** built the SUMO mobility and

LuST scenario, the ns-3 802.11p network, the ledger bridge and the integration of all parts; **Harsh Kumar Gour** the Hyperledger Fabric network, the chaincode and the REST gateway; **Byreddy Varshith Reddy** the authentication protocol (zero-knowledge login, tickets, handover and revocation); and **Andela Harsha Vardhan** the misbehaviour detection and the evaluation tooling.

1.6 Report Structure

Chapter 2 covers the building blocks and the threat model, Chapter 3 related work, Chapters 4–5 the scheme, its optimisations and the implementation, Chapter 6 the evaluation and Chapter 7 the conclusions. The appendix explains how to reproduce every result.

Chapter 2

Background and Problem Statement

This chapter fixes the security requirements, the threat model and the building blocks the scheme uses, and ends with the problem statement.

2.1 Security Requirements

1. **Authentication and integrity:** a receiver accepts a message only if it comes from a vehicle registered by the TA and has not been altered.
2. **Anonymity:** neither RSUs nor other vehicles learn a vehicle's real identity from authentication or safety messages.
3. **Unlinkability:** an observer should not be able to link the pseudonyms of one vehicle over time; at least, changing pseudonyms must be possible and effective.
4. **Conditional traceability:** an authorized investigation can reveal the identity behind a pseudonym, but no single party can do so alone.
5. **Revocation:** a misbehaving pseudonym, and after an investigation the whole vehicle, can be excluded quickly from the network.
6. **Authorization:** an authenticated vehicle may still be denied a service its class is not entitled to (for example, a private car asking for emergency priority).
7. **Replay resistance:** an old proof or message cannot be accepted again.
8. **Performance:** authentication within a few milliseconds and safety-message overhead small enough to keep the shared channel uncongested.

2.2 Threat Model

- **Outsider attackers** can listen to every message (a global passive eavesdropper), inject, replay and modify messages, but hold no registered credential.
- **Insider attackers** are registered vehicles that send false information: false positions or speeds, sudden-stop claims, message floods, and, in the emergency extension, false emergency claims. They may use all of their own pseudonyms.
- **RSUs** follow the protocol but are curious: they try to learn identities. Up to f of the $n = 3f + 1$ RSU organizations may be compromised and report honest vehicles; the trust-weighted extension (Section 4.9.7) addresses this case.
- **The TA** registers vehicles and is trusted for registration. It cannot unmask a pseudonym alone: identity records are escrowed under a key shared among the RSU organizations.
- **The ledger** is a permissioned blockchain whose ordering service tolerates crash faults (Raft) or Byzantine faults (SmartBFT).

Physical attacks on OBUs, radio jamming and denial of service by exhausting the radio channel are outside the scope.

2.3 Cryptographic Building Blocks

All public-key operations use the elliptic curve secp256k1 [13] with generator G of prime order q, and the hash function H is SHA-256 [14].

2.3.1 Schnorr Proof of Knowledge

A vehicle holds a secret key $x \in \mathbb{Z}_q$ for each pseudonym, with public key $Y = xG$. The Schnorr protocol [15], made non-interactive with the Fiat–Shamir transform [16],

proves knowledge of x without revealing it:

$$\text{commit:} \quad r \xleftarrow{\$} \mathbb{Z}_q, \quad T = rG \tag{2.1}$$

$$\text{challenge:} \quad c = H(G \,\|\, Y \,\|\, T \,\|\, ctx) \tag{2.2}$$

$$\text{response:} \quad s = r + c\,x \bmod q \tag{2.3}$$

$$\text{verify:} \quad sG \stackrel{?}{=} T + cY \tag{2.4}$$

The context ctx binds the proof to one RSU, one fresh RSU nonce and timestamp, the requested service, the pseudonym and the vehicle class, so a proof cannot be replayed at another RSU, later, or for another service. The proof is *sound* (without x a forger succeeds with probability about $1/q$) and *zero-knowledge* (a simulator that picks c, s first and sets $T = sG - cY$ produces proofs with the same distribution, so a real proof reveals nothing about x). The commitment T does not depend on the RSU, so a vehicle can precompute a pool of commitments while idle and only compute s when it meets an RSU.

2.3.2 Merkle Trees

A Merkle tree [17] hashes a list of leaves pairwise up to a single root. A path of $\lceil \log_2 N \rceil$ sibling hashes proves that a leaf is in the tree. The TA builds one tree per epoch over the leaves

$$\ell = \mathsf{enc}(\texttt{"VANET-LEAF-v1"},\ pid,\ Y,\ class),$$

where enc length-prefixes each field. Only the 32-byte root goes on the ledger. Because the pseudonym, its key and the vehicle class are inside the leaf, a vehicle cannot claim another pseudonym or class than the TA registered: the path would not verify.

2.3.3 Threshold Identity Escrow

The TA encrypts each vehicle's identity record (its real identity and its list of pseudonyms) with AES-256 in GCM mode and splits the key with Shamir's (t, n) secret sharing [18] among the n RSU organizations, with $t = 2f + 1$. An RSU releases its share only for a trace request that the ledger shows as approved. Any t shares reconstruct the key

by Lagrange interpolation; fewer reveal nothing.

2.3.4 Signatures, TESLA and Implicit Certificates

Tickets and handover requests use Schnorr signatures. The compact wire format uses BIP-340 [19] 64-byte signatures with 32-byte x-only keys.

TESLA [20] replaces a signature per message with a message authentication code (MAC) under a key from a one-way hash chain [21] $K_i = H(K_{i+1})$. The sender uses K_i during interval i and discloses it d intervals later. A receiver buffers the message, checks the disclosed key against the signed chain commitment, and then the MAC. A MAC is 16 bytes and about ten times cheaper to compute than a signature; the cost is a delay of about d intervals before a message can be accepted.

An ECQV implicit certificate [22] replaces the RSU's signature on a ticket: the certificate carries a point P_U from which any receiver reconstructs the holder's public key as $Q_U = H(cert)\,P_U + Q_{CA}$. A certificate is about 64 bytes smaller than an explicit key plus signature, and it is authenticated implicitly by the first signature verified under Q_U.

2.3.5 Bloom Filters

A Bloom filter [23] represents a set in m bits with k hash functions. It has no false negatives and a tunable false-positive rate. The extension uses one to put the recently revoked pseudonyms into RSU beacons.

2.4 Permissioned Blockchain: Hyperledger Fabric

Hyperledger Fabric [24] is a permissioned blockchain in which every participant has a certificate from its organization's Membership Service Provider (MSP). A transaction is executed by the endorsing peers (*execute*), ordered into blocks by the ordering service (*order*) and validated and committed by every peer (*validate*). Smart contracts, called chaincode, can read the caller's identity and therefore enforce role-based rules: only the TA may register roots or policies, and only RSUs may record authentications or report misbehaviour.

Blocks. Block i holds m transactions and a header that commits to them and to the previous header:

$$D_i = H(tx_{i,1} \,\|\, \ldots \,\|\, tx_{i,m}), \qquad h_i = H(i \,\|\, h_{i-1} \,\|\, D_i). \tag{2.5}$$

Changing any recorded transaction changes D_i, hence h_i and every later header, so a peer that holds h_j for some $j \geq i$ detects the change.

Validation. A transaction T carries the endorsements $E(T)$ of the peers that executed it and the read set $RS(T)$ of keys and versions it read. Every peer commits it only if

$$\mathrm{EP}\big(E(T)\big) = \text{true} \;\wedge\; \forall (k, v) \in RS(T) : \; \mathrm{ver}(k) = v, \tag{2.6}$$

that is, if the endorsements satisfy the chaincode's endorsement policy EP (here the default, a majority of the peer organizations) and no key it read has changed since (multi-version concurrency control). Otherwise the transaction stays in the block but is marked invalid.

Ordering. The ordering service runs either Raft [25] or SmartBFT [26]. Raft commits a block once a majority of the n orderers have stored it and so tolerates f crashed orderers for $n = 2f + 1$. SmartBFT tolerates f Byzantine orderers for $n \geq 3f + 1$ [27, 28] and commits after $2f + 1$ matching votes; the deployment uses $n = 4$, $f = 1$.

Commit latency. A write becomes visible after

$$T_{\text{commit}} = T_{\text{endorse}} + T_{\text{batch}} + T_{\text{order}} + T_{\text{validate}}, \qquad 0 \leq T_{\text{batch}} \leq \tau, \tag{2.7}$$

where T_{batch} is the wait until the orderer cuts the block. While blocks fill more slowly than their maximum size, a block is cut τ after its first transaction (the batch timeout), so τ dominates at low load.

2.5 Misbehaviour Detection

Cryptography proves who sent a message, not that its content is true: a registered vehicle can still broadcast a false position. Misbehaviour detection checks plausibility: a vehicle cannot move faster than physically possible, accelerate beyond a limit, or be

heard from beyond radio range. The VeReMi dataset [29] and its extension [30] are the standard benchmark: realistic traces with 19 labelled attack types, among them constant or random position and speed offsets, sudden stops, replay, denial of service and Sybil variants.

2.6 Problem Statement

Design a VANET authentication and authorization scheme that

1. proves registration and vehicle class with a zero-knowledge proof bound to the RSU, the time, and the service, so that verifiers learn neither the identity nor the secret;
2. keeps authorization, revocation and identity-reveal approvals on a permissioned blockchain without placing the blockchain on the path of individual safety messages;
3. removes misbehaving vehicles with conditional traceability that no single party controls;

and evaluate on a realistic 802.11p network with real traffic whether its latency, channel load and detection performance are acceptable compared with certificate-based and centralized baselines.

Chapter 3

Literature Review

VANET authentication schemes optimise different objectives: privacy, efficiency, decentralisation, accountability or support for mobility. This chapter groups them by approach, notes what each contributes and what it leaves open for this work, and positions the project.

3.1 PKI and Pseudonymous Certificates

Raya and Hubaux [6] define the VANET threat model and a public-key infrastructure in which every vehicle carries a large pool of short-lived anonymous certificates. The authority can resolve a pseudonym when an investigation is authorised. This architecture underlies the US Security Credential Management System (SCMS) [7] and IEEE 1609.2 [31]. Its costs are the pre-loading and refresh of thousands of certificates and large certificate revocation lists. Privacy comes only from changing identifiers, and the authority stays on the trust path. Zhong et al. [32] compress many signatures into one with certificateless aggregate signatures, which cuts verification and communication, but the trace authority remains a single point of trust and no ledger shares revocation state between RSUs.

3.2 ECC and Certificateless Authentication

Liu et al. [33] propose cooperative ECC-based mutual authentication with pseudonyms, batch verification and group keys for low RSU workload. The trusted authority issues and holds every long-term key, so it can link any pseudonym to a real identity. Qi et al. [34] combine pseudonyms with certificateless signatures to avoid certificate management and key escrow. Certificateless cryptography removes the certificate,

not the authority: a key generation centre still issues partial keys, and per-message verification remains.

3.3 Hardware and Game-Theoretic Approaches

Li et al. [35] derive authentication secrets from physical unclonable functions (PUFs), which resist key extraction but require PUF hardware in every OBU and cannot be retrofitted to a mixed fleet. Gupta et al. [36] combine a game-theoretic trust model with a blockchain to decide authentication, at the cost of decision and consensus complexity.

3.4 Blockchain-Based Authentication

Blockchains give RSUs shared, tamper-evident state. BAAIoV [10] uses a blockchain for both authentication and authorization in the Internet of Vehicles, and BPA [11] builds privacy-preserving authentication on it. B-TSCA [12] uses blockchain-assisted trustworthiness computation so that re-authentication at the next RSU is cheaper than the first authentication. Xie et al. [37] add PUFs for lightweight V2I handover and V2V broadcast authentication. In all of these the ledger supplies shared state, but the credential is still shown to the verifier, so privacy depends on pseudonym management and nothing prevents linking two sessions of the same vehicle.

3.5 Zero-Knowledge Authentication

Rasheed et al. [8] use group-based zero-knowledge proofs with configurable privacy and revocation. They rely on interactive proof rounds and a trusted group manager, which costs round trips at every handover. ZAMA [9] combines a ZKP, ECC and a Fujisaki–Okamoto commitment for anonymous mutual authentication with traceability, at a higher cost per authentication and with a central authority. Varma and Kumar [38] focus on lightweight ZKP authentication during handover. Without decentralised state, authorization and revocation decisions cannot follow a vehicle between RSUs and cannot be audited independently.

3.6 Blockchain with Zero-Knowledge Proofs

The schemes closest to this work combine both. Jiang and Guo [39] use master–slave chains with a PBFT variant and a ZKP. Wang et al. [40] use identity-based polynomial commitments, a ZKP and a Merkle Patricia trie on a blockchain for anonymous authentication and data retrieval; a key generation centre remains in the setup, and service-level policy, the revocation lifecycle and RSU handover are not modelled. Li et al. [41] authenticate emergency vehicles with one-time hash commitments, category Merkle trees, a ZKP and on-chain state, but cover a small static fleet under one authorization decision.

3.7 Misbehaviour Detection and Location Privacy

Plausibility checks and learned detectors are evaluated on VeReMi [29] and VeReMi Extension [30]. For privacy against a global eavesdropper, pseudonym changes alone are weak because a tracker links the old and the new pseudonym by position and speed [5]; mix zones [42, 43], where vehicles change pseudonyms together and stay silent briefly, break that link.

3.8 Comparison and Research Gap

Table 3.1: Families of VANET authentication schemes (adapted from the Phase I review)

Family	Identity privacy	Shared state	Accountability	Main limitation
PKI / pseudonym [6]	medium–high	low	high	certificates and revocation lists
ECC / certificateless [33, 34]	medium	low	high	dependence on the TA
Aggregate [32]	high	low	medium–high	trace authority
PUF [35]	high	low	medium	special hardware
Blockchain [10, 11, 12, 37]	medium	high	high	consensus latency
ZKP [38, 8, 9]	very high	low	medium–high	proof cost, no shared state
Blockchain + ZKP [39, 40, 41]	very high	high	high	integration, scalability
This work	very high	high	high	measured in Chapter 6

Table 3.1 summarises the families. The direct blockchain-plus-ZKP literature shows that the combination is feasible. What it does not provide is a complete lifecycle (handover, renewal, misbehaviour reporting, revocation that reaches vehicles, threshold identity reveal, per-service policy) evaluated on a realistic network with

measured costs. This project therefore does not claim the combination as new. Its contribution is a concrete integration that keeps the ledger off the safety-message path, and its measurement against certificate-based and centralised baselines on ns-3 with real traffic.

Chapter 4

Proposed Scheme

This chapter describes the core of the scheme: registration, authentication, handover, detection, revocation and identity reveal, then the mechanisms that make it cheap on the radio channel and robust (Section 4.9) and its support for emergency vehicles (Section 4.10). Together they are the scheme evaluated in Chapter 6.

4.1 Architecture and Design Principle

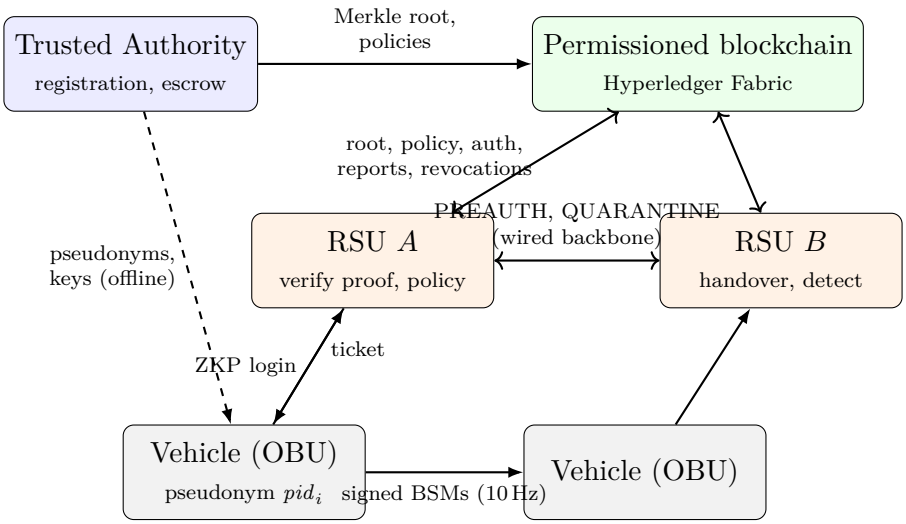


Figure 4.1: Entities and the information each exchanges. The blockchain holds only state that several RSUs must share and that must be auditable; it is not on the path of individual safety messages.

Figure 4.1 shows the entities. The guiding principle of the design is that *proofs are verified at the edge and the ledger stores only shared state.* An RSU verifies a zero-knowledge proof locally in well under a millisecond, while a Fabric commit takes hundreds of milliseconds (Section 6.6). So the ledger is read from a local, regularly synchronised copy, written asynchronously, and never waited for while a safety message

is handled. The ledger stores:

- the Merkle root of each registration epoch (never a per-vehicle record);
- service policies: which vehicle classes may use which service;
- authorization records keyed by pseudonym, so that the next RSU can accept a handover;
- misbehaviour reports with evidence hashes, and the revocation list;
- trace requests and the RSU organizations' approvals for identity reveal.

4.2 Registration

For each vehicle v the TA generates P pseudonyms (5 by default), each a fresh key pair $(x_i, Y_i = x_i G)$ with a random 64-bit identifier pid_i, and the vehicle's class (*car*, *truck* or *emergency*). It builds the Merkle tree over the leaves of all pseudonyms of all vehicles, in shuffled order so that positions do not reveal which pseudonyms belong together, and gives each vehicle its keys and Merkle paths over a secure channel. It publishes the root with its validity period on the ledger through the chaincode function `RegisterRoot`. Finally it encrypts the vehicle's identity record (identity and list of pseudonyms) and escrows the key with the RSU organizations as t-of-n Shamir shares, with $t = 2f + 1$ for $n = 3f + 1$ organizations (5 of 9 in the evaluated scenarios).

4.3 Authentication

Figure 4.2 shows the exchange.

1. Every RSU broadcasts a signed beacon every 100 ms with its identity, a fresh nonce n and a timestamp. A vehicle accepts only beacons from RSUs whose public keys it holds.
2. The vehicle computes the Schnorr response for the current pseudonym with

$$ctx = \mathsf{enc}(\texttt{"VANET-AUTH-v1"}, rsu, n, ts, svc, pid, class)$$

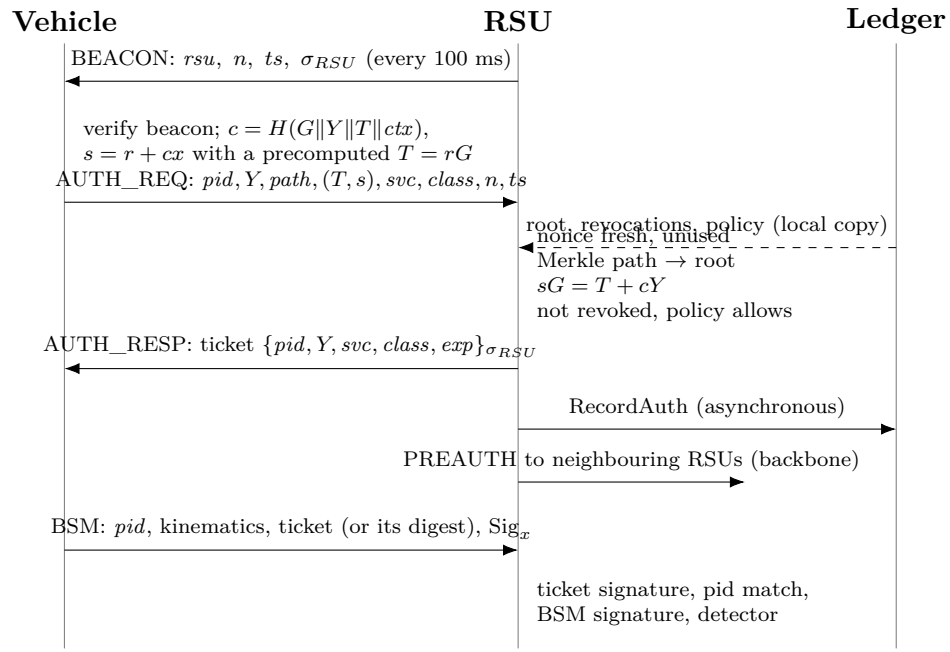


Figure 4.2: Authentication, ticket issue and the first safety message

and sends the pseudonym, its public key, the Merkle path, the proof, the requested service and its class.

3. The RSU checks that the nonce is one it issued recently and that this (nonce, pseudonym) pair was not used before, rebuilds the leaf and verifies the path against the current root from the ledger, verifies the proof, checks that the pseudonym is neither revoked nor quarantined, and evaluates the authorization predicate

$$\text{Access}(v, svc) = 1 \iff \text{Verify}(\pi, Y) = 1 \ \wedge \ pid \notin RL \ \wedge \ class \in \text{Policy}(svc). \tag{4.1}$$

4. If access is granted, the RSU signs a ticket over the pseudonym, its key, the service, the class, its own identity and an expiry 30 s ahead, records the authorization on the ledger asynchronously, and sends a signed PREAUTH note over the backbone to its neighbouring RSUs.

The RSU learns the pseudonym, the class and that the vehicle is registered, and nothing else: the leaf of another vehicle of the same class looks the same, the proof is zero-knowledge, and the Merkle path reveals only the leaf position, which is shuffled.

4.4 Handover and Pseudonym Change

When a vehicle approaches the edge of its RSU's range (80 % of the nominal 300 m) and hears a closer RSU, or loses its RSU, it sends a handover request with its ticket, signed with the ticket key and bound to the new RSU's beacon nonce. The new RSU checks the issuing RSU's signature on the ticket, the request signature and the shared authorization: either the authorization record on the ledger or, if the commit has not reached its local copy yet, the PREAUTH note from the issuing RSU. A handover needs no new proof. After three sessions a vehicle switches to its next pseudonym and authenticates afresh, so that a pseudonym's lifetime stays short.

4.5 Safety Messages

Each BSM carries the pseudonym, the kinematic fields and the ticket, and is signed with the pseudonym's secret key, whose public key is in the ticket. A receiver checks the ticket's RSU signature, that the ticket's pseudonym matches, the BSM signature and the ticket expiry. The receiver needs no certificate chain and no ledger access. Carrying the full ticket would make every BSM about 330 bytes; Section 4.9 replaces it by a digest and most signatures by TESLA MACs.

4.6 Misbehaviour Detection and Reporting

Every RSU runs a detector on each authentic BSM against that pseudonym's recent history. Hard plausibility rules flag positions beyond radio range, speeds above 60 m/s, accelerations above the physical limit, positions inconsistent with the claimed speed, impossible heading flips, replays and message rates above 13 per second. Weak rules (sudden appearance close to the RSU, two pseudonyms at the same place) count only in pairs. An RSU reports a pseudonym once $K = 3$ of its last $W = 10$ messages were flagged, with the hash of a flagged, *signed* message as evidence, through the chaincode function `ReportMisbehaviour`.

4.7 Revocation and Threshold Identity Reveal

The chaincode revokes a pseudonym automatically once two distinct RSUs have reported it. A single report, perhaps a detector error, therefore has no effect. RSUs see the revocation through their local ledger copy and reject the pseudonym's BSMs and logins. An RSU that reported a now-revoked pseudonym opens a trace request on the ledger (`RequestTrace`), and the RSU organizations approve it (`ApproveTrace`). Once t approvals are recorded, each approving organization releases its key share, the TA reconstructs the escrow key, decrypts the identity record and revokes all of the vehicle's pseudonyms. The vehicle's next login attempt fails and it is shut out. No single party can unmask a vehicle: the TA lacks the key, and fewer than t RSU organizations hold too few shares.

4.8 Ledger Interface

Table 4.1: Chaincode functions and who may call them

Function	Caller	Effect
`RegisterRoot`	TA	publish the Merkle root of an epoch
`SetPolicy`, `SetConfig`	TA	classes allowed per service; K, trace threshold
`RecordAuth`	RSU	authorization record of a pseudonym (rejected if revoked)
`ReportMisbehaviour`	RSU	report with evidence hash; revoke at K reporters
`Revoke`	TA	revoke pseudonyms after a trace
`RequestTrace`, `ApproveTrace`	RSU	open and approve an identity reveal
`SetReporterWeight`	TA	report weight of an RSU (Section 4.9.7)
`IsRevoked`, `GetRoot`, …	any	queries; `ListRevokedSince` for synchronisation

Table 4.1 lists the chaincode functions. The chaincode reads the caller's role from its Fabric certificate, so an RSU cannot change a policy and a vehicle cannot write at all. RSUs reach Fabric through a REST gateway that signs each transaction with the caller's identity.

Ledger state and rules. The world state holds the Merkle roots, the policies $\mathrm{Pol}(svc)$, the authorization records, for every pseudonym p the set $\mathcal{R}(p)$ of RSUs that reported it (one report per RSU, stored under the key (p, u)), the revocation set $\mathcal{V}$,

and for every trace request q the set of approving organizations $A(q)$. With u the calling RSU, the chaincode accepts an authorization record (`RecordAuth`) and revokes and reveals according to

$$\text{accept}(p, svc, c) \iff \text{role}(u) = \text{RSU} \ \wedge \ p \notin \mathcal{V} \ \wedge \ c \in \text{Pol}(svc), \tag{4.2}$$

$$\text{revoke}(p) \iff \textstyle\sum_{u \in \mathcal{R}(p)} w_u \geq K, \tag{4.3}$$

$$q \text{ approved} \iff |A(q)| \geq t. \tag{4.4}$$

Each weight $w_u \in [0, 1]$ is 1 unless trust weighting has lowered it (Section 4.9.7), so by default (4.3) is $|\mathcal{R}(p)| \geq K$ distinct RSUs. A revoked pseudonym enters $\mathcal{V}$, as do all pseudonyms of a vehicle that the TA revokes after an approved trace. No chaincode function removes a pseudonym from $\mathcal{V}$, so a revocation is permanent, and every change to $\mathcal{R}$, $\mathcal{V}$ and A is a signed transaction in the hash chain of Section 2.4. An RSU's decision to admit a vehicle (Section 4.3) checks the same predicate as (4.2) on its local copy of the state, which is why the ledger can stay off the login path.

4.9 Efficiency and Robustness Mechanisms

The core scheme described so far would be costly on the radio channel: every BSM would carry a full ticket and a signature of about 330 bytes, ten times a second, and vehicles would have no revocation information at all, accepting messages from revoked senders until their tickets expired. This section describes the mechanisms that complete the scheme. Table 4.2 lists them with what each improves.

Table 4.2: Mechanisms added to the core scheme

Mechanism	Improves
Ticket digests, compact wire format, handover hysteresis	message size, handovers
Fast quarantine over the RSU backbone	time until an attacker is ignored
TESLA with adaptive signing and key messages	safety-message size
ECQV implicit-certificate tickets	ticket size
RSU clusters (2×2)	number of handovers
Load-triggered batch verification	RSU and vehicle processor load
Dead-reckoning BSM triggering	number of safety messages
Revocation filter in beacons	vehicles ignore revoked senders
Trust-weighted reports	robustness against compromised RSUs
Ticket renewal; signal-strength check	login load; position spoofing

4.9.1 Smaller Messages: Digests, Compact Encoding, Implicit Certificates

A vehicle sends its full ticket only every tenth BSM, after hearing a new RSU, and at least every 2 s; the other BSMs carry a 16-byte ticket digest that receivers resolve from a cache. The compact wire format packs the kinematics into 16 bytes and uses 64-byte BIP-340 signatures. With ECQV (Section 2.3.4) the RSU issues an implicit certificate instead of signing the ticket, which saves the explicit key and signature. The certificate request point is bound into the proof context, so nobody can swap it on the air and receive a certificate for someone else's pseudonym.

4.9.2 Cheaper Authentication: TESLA with Adaptive Signing

With TESLA a BSM carries a 16-byte MAC under the current chain key instead of a signature. Keys are disclosed two 25 ms intervals later, in the vehicle's next BSM or in a small key message, so a receiver can accept the BSM after about 50 ms. A signed anchor BSM carries the chain commitment. To avoid the TESLA delay when it brings nothing, the vehicle signs every BSM while its channel is quiet and switches to TESLA when the channel busy ratio exceeds 0.30 (and back below 0.20, with 1 s of hysteresis). Urgent BSMs, after braking harder than $3\,\mathrm{m/s^2}$, are always signed so that neighbours can act on them at once.

4.9.3 Fewer Messages: Dead Reckoning

Receivers predict a vehicle's position from its last BSM by dead reckoning. The vehicle sends a BSM only when that prediction drifts by more than 1 m, its speed changes by more than 0.5 m/s or its heading by more than 4°, and at least once a second. This follows the generation rules of the European cooperative awareness service [2]. On straight roads at constant speed most BSMs become unnecessary, while every manoeuvre is still reported immediately.

4.9.4 Fewer Handovers: RSU Clusters and Ticket Renewal

RSUs are grouped into clusters of 2×2, and a ticket is valid at every RSU of its cluster, so moving between them needs no messages at all. Before a ticket expires, the vehicle renews it with one signed request instead of a new proof.

4.9.5 Load-Triggered Batch Verification

Schnorr signatures can be verified in batches with one multi-scalar multiplication [44]. The measured speed-up is 1.75× for 8 signatures and 1.97× for 32. Waiting to fill a batch would add latency, so batching is load-triggered: a signature that finds the processor idle is verified at once, and only signatures that arrive while it is busy are verified together, up to 32, the moment it is free. If a batch fails, bisection finds the invalid signatures.

4.9.6 Revocation that Reaches Vehicles

Revocation filter. Every fifth beacon (2 Hz) carries a Bloom filter (7 hash functions, about 1 % false positives) of the pseudonyms revoked or quarantined within the last ticket lifetime. Vehicles drop BSMs from pseudonyms in the filter of the last beacon they verified. False positives are possible but rare, and they only drop a message.

Fast quarantine. An RSU that reports a pseudonym also sends a signed QUARANTINE note over the backbone to its neighbouring RSUs. Once notes from $K = 2$ distinct RSUs arrive, they block the pseudonym locally until the ledger revocation lands, which takes milliseconds instead of a ledger commit.

4.9.7 Trust-Weighted Reports

A compromised RSU could report honest vehicles. With trust weighting, the TA investigates every traced pseudonym. In the simulation the ground truth stands in for the investigation. Each RSU that reported it gets a weight

$$w = \min\left(1,\ \frac{2\,(tp+1)}{tp+fp+2}\right), \tag{4.5}$$

where *tp* and *fp* count its confirmed and wrong reports. This is twice the expected trust of the beta reputation system [45], capped at 1. Revocation then needs a report weight of at least K instead of K reporters. An honest RSU keeps weight 1; an RSU with more wrong than right reports loses influence. Weights are stored on the ledger through `SetReporterWeight`.

4.9.8 Signal-Strength Check

An RSU estimates the sender's distance from the received power using the scenario's path-loss model with 4 dB log-normal shadowing. It flags a BSM whose claimed position disagrees with that estimate by more than four standard deviations and by at least 100 m. A position-forging attacker cannot move its own transmitter, so its claims stop matching its signal.

4.10 Emergency Vehicles

Emergency vehicles need the opposite of anonymity on one point: their privileged status must be verifiable by everyone, and must be impossible to fake. The extension builds this on the authorization predicate (4.1).

1. **Registration.** The TA registers an ambulance's pseudonyms with class *emergency*. The class is in the Merkle leaf and in the proof context, so a car cannot claim it.

2. **Service.** Ambulances log in for the service `emergency-priority`, whose ledger policy admits only class *emergency*. Their tickets therefore carry, signed by the RSU, the class and the service.

3. **Siren alerts.** With the siren on, every BSM of an emergency vehicle is signed (never TESLA, so receivers need not wait), carries the full ticket and a flag `ev` inside the signed payload, and is never suppressed by dead reckoning. Vehicles process alerts from up to 300 m, beyond the 100 m safety radius of normal BSMs.

4. **Verification.** A receiver honours the flag only if the RSU-signed class of the sender's ticket is *emergency*; otherwise it rejects the message as a fake. A signed fake is itself proof of the lie: no detector judgment is involved. The first RSU

that receives one reports the pseudonym at once with the message hash as evidence, and quarantine and revocation follow.

5. **Priority.** Emergency logins, renewals and handovers, and the RSUs' answers to them, are sent on the 802.11p access category AC_VI instead of AC_BE, so they do not wait behind the queue of ordinary logins on the radio. Safety messages keep the highest category, AC_VO. The RSU also verifies them ahead of its processing queue: such a login waits only for the job in progress. Because the claim is checked only by the verification itself, an RSU grants priority to at most five claims per second, so false claims cannot crowd out everyone else.

The fake-emergency attacker of the evaluation tries one emergency login, which the policy denies, and then sets the flag on all of its BSMs.

4.11 Security Analysis

- **Impersonation.** An accepting proof requires x_i for a leaf in the registered tree (soundness of the Schnorr proof and collision resistance of the hash). A signed BSM requires the key in an RSU-signed ticket.
- **Replay.** A login proof is bound to the RSU, a nonce the RSU issued within the freshness window and the pseudonym, and each (nonce, pseudonym) pair is accepted once. BSMs carry a timestamp and sequence number that the detector checks; handover requests are bound to the new RSU's nonce.
- **Class and service escalation.** The class is part of the Merkle leaf and of the proof context, so claiming another class breaks verification; the policy check (4.1) then decides the service.
- **Anonymity.** RSUs and vehicles see pseudonyms only, and the proof is zero-knowledge. Linking pseudonyms of one vehicle requires its escrow record, which needs t organizations.
- **Accountability.** Every revocation and every reveal is a ledger transaction signed by identified organizations, and the ledger is append-only.
- **Limitations.** The TA knows at registration which pseudonyms belong to which

vehicle. The RSUs learn which pseudonym was where, and a global eavesdropper can link consecutive pseudonyms by position. Anonymous credentials and mix zones would address both; they are left to future work (Chapter 7).

Chapter 5

Implementation

The system is implemented in Python (protocol, simulation glue, detection and evaluation), Go (chaincode) and JavaScript (ledger gateway and live map). Everything is in one repository and runs from a Makefile. This chapter describes the parts and how they connect.

5.1 Overview

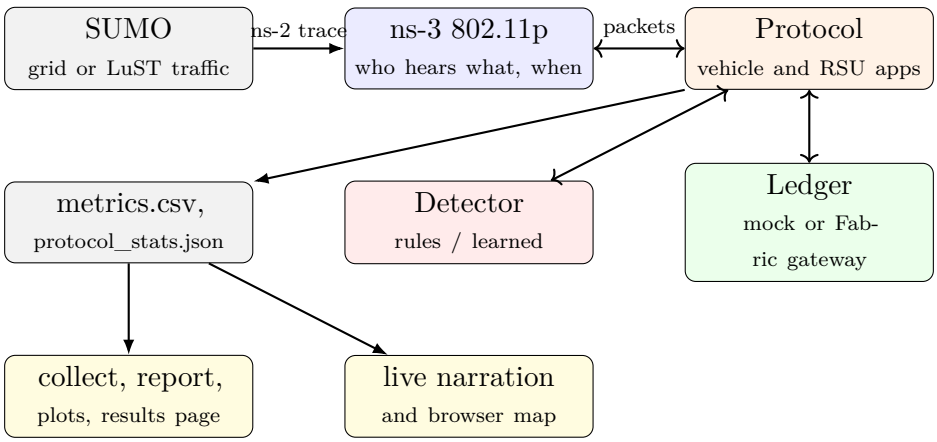


Figure 5.1: Simulation and evaluation pipeline

Figure 5.1 shows the pipeline. SUMO [46] decides where every vehicle drives. ns-3 models the radio: transmission, propagation, collisions and the MAC. The protocol code runs inside every vehicle and RSU node and talks to the ledger and the detector. Every event is logged to `metrics.csv`, from which the evaluation computes all results. Table 5.1 lists the main parts of the code.

Table 5.1: Code organisation

Folder	Contents
`mobility/`	grid and LuST scenario builders, SUMO-to-ns-2 trace conversion, RSU placement
`network/`	scenario runner, ns-3 radio backend, node hooks, metrics, ledger bridge, live map
`protocol/`	vehicle and RSU logic, cryptography, message encoding, registration, escrow, TESLA, ECQV, Bloom filter, batch verification, V2V verifier, simulator glue
`ledger/`	Fabric network scripts, Go chaincode, Node.js REST gateway, mock ledger, conformance tests, load benchmark
`detection/`	detector (rules, features, learned models), VeReMi loaders, benchmark and training
`experiments/`	sweep configurations, runner, collection, reports, plots, results page, tracker attack

5.2 Mobility

Grid. A 5×5 junction grid with 250 m blocks ($1\,\mathrm{km}^2$) and two lanes per direction. Random trips start during the first half of the run, so vehicles enter one after another. Nine RSUs stand 450 m apart, each with a nominal range of 300 m, so ranges overlap.

LuST. The Luxembourg SUMO Traffic scenario [47] models a full day of the city's traffic. The evaluation simulates the whole city and cuts a $1.35\,\mathrm{km} \times 1.35\,\mathrm{km}$ box in the centre at three times of day: 06:00 (about 150 vehicles in the box), 14:00 (about 260) and the 08:00 peak (about 525). A 900 s warm-up fills the roads before the measured window. Vehicles that leave the box are parked outside radio range until they come back.

Mobility step and lane changes. SUMO runs at 0.1 s steps. Its default instant lane change is a 3.2 m sideways jump in one step, which the trace turned into a false 32 m/s velocity that the detector rightly flagged. The evaluation therefore uses a 2 s lane-change duration, as real drivers do, which removed all such spikes on the grid and all but two of 385 on LuST.

5.3 Radio Network

ns-3 (version 3.44 [48]) is driven from Python through its cppyy bindings. Table 5.2 lists the radio configuration. A lighter pure-Python backend with the same parameters

Table 5.2: IEEE 802.11p parameters in ns-3

Parameter	Value
Standard, band	IEEE 802.11p (WAVE), 5.9 GHz, 10 MHz channel
Data rate	6 Mb/s
Transmit power, receiver sensitivity	20 dBm, −85 dBm
Propagation	log-distance, exponent 2.196, 47.86 dB at 1 m
Nominal range	400 m
Access categories	BSMs, beacons and TESLA keys on AC_VO; the rest on AC_BE
Wired RSU backbone	2 ms latency

exists for development and unit tests; every reported result comes from ns-3. ns-3's FlowMonitor and the PHY's busy time give the delivery ratio and the channel busy ratio, sampled every second at each RSU.

5.4 Protocol

The protocol package implements Chapter 4. Schnorr signatures and proofs use `coincurve` (libsecp256k1), and the escrow uses AES-GCM [49] from `cryptography`. Messages use a compact binary encoding: one byte per field name from an append-only table, binary hex strings, and 16-byte packed kinematics. Vehicles and RSUs are applications attached to ns-3 nodes through a small set of hooks: send, schedule, position, log and the local ledger copy.

Processing time. Simulated time does not advance while Python computes. Instead, the cost of every cryptographic operation was measured on the real implementations (Table 5.3) and is charged as processing delay. With *per-node processor* modelling, each RSU and vehicle has one processor that executes this work in arrival order. Latency therefore includes queueing, and a BSM that would wait more than 100 ms is dropped as too old to be useful. Vehicle processors are assumed five times slower than the development PC, an AMD Ryzen 7 4800H. A sensitivity sweep varies this factor (Section 6.7).

Vehicle-side verification. Each vehicle verifies the BSMs of senders within 100 m on its own processor, as a receiver that acts on them would. It caches tickets learned from full-ticket BSMs of farther senders.

Table 5.3: Measured cost of each cryptographic operation on the development PC (ms)

Operation	Time (ms)
Schnorr proof (vehicle, with precomputed commitment)	0.040
Schnorr proof verification (RSU)	0.077
Merkle path check	0.006
Schnorr signature	0.082
Schnorr signature verification	0.076
BSM signature verification	0.082
TESLA MAC	0.006
ECQV certificate issue	0.056
ECQV key reconstruction	0.048
ECDSA P-256 sign (baseline)	0.022
ECDSA P-256 verify (baseline)	0.058

5.5 Ledger

The chaincode is written in Go for Hyperledger Fabric 3.1.5 and deployed on a Fabric network with two peer organizations and an ordering service of one Raft orderer or four SmartBFT orderers. A Node.js gateway exposes it as a REST interface. It maps an `X-Identity` header to the caller's Fabric certificate, signs transactions, reports endorsement and commit times, and keeps an activity log. An HTTP conformance suite runs unchanged against the real gateway and against a mock with the same rules. For long parameter sweeps, the simulator uses an in-process ledger with the same functions and a 500 ms commit delay, which is the Raft commit time measured with the default batch timeout (Section 6.6). RSUs read from a local copy that is updated every second, or at the mock's commit time.

5.6 Detection

The detector computes per-message plausibility features against the pseudonym's history: implied speed, speed and position consistency over 0.8–3 s baselines, acceleration, heading change, message rate, distance from the RSU, the signal-strength distance and sequence numbers. It applies the rules of Section 4.6. For comparison, learned models over the same features (a random forest and gradient-boosted trees [50] with a history window, built with scikit-learn [51]) are trained and tested on VeReMi Extension [30], on five vehicle-disjoint splits so that no vehicle's messages appear in both

training and test data. The scheme itself uses the rules and the signal-strength check, which do not depend on the road network.

5.7 Evaluation Framework

A sweep is a JSON configuration: fixed options, a grid of parameters (for example 50, 100 and 150 vehicles), the variants to compare and the seeds. The runner executes every combination in parallel and skips finished runs. A collection step turns each run's event log into about 100 metrics. The report step computes means with 95 % confidence intervals over the seeds and Welch's t-test [52] against each baseline, and writes Markdown and LaTeX tables, PNG, PDF and SVG figures, and an interactive results page. The tables and numbers in this report are generated from the same outputs (Appendix A).

Baselines. Two baselines run on the same simulator. *ECDSA-PKI* represents IEEE 1609.2-style pseudonym certificates: the vehicle presents a CA-signed certificate and signs with ECDSA [53] P-256, using the measured ECDSA costs. *Central-TA* sends every login over the backbone to one central authority that decides and answers.

Tracker. For privacy, a global eavesdropper records every BSM. A tracker then tries to link each new pseudonym to a disappearing one by predicting positions from the last speed and heading. The metrics are the share of pseudonym changes linked correctly and the share of a trip that can be followed from start to end.

5.8 Live Demonstration Tools

Two tools show a run as it happens. A terminal narration prints each important event in plain language, with a one-line explanation the first time each kind appears, and a status line every five simulated seconds. A live map in the browser (Figure 5.2) draws the SUMO road network, the RSUs with their coverage and every vehicle, coloured by its state. Clicking a vehicle shows its pseudonym, class, ticket, revocation state and recent events. Clicking an RSU shows its load, the vehicles in range, its counters and its recent checks. The page can pause the simulation and change its speed.

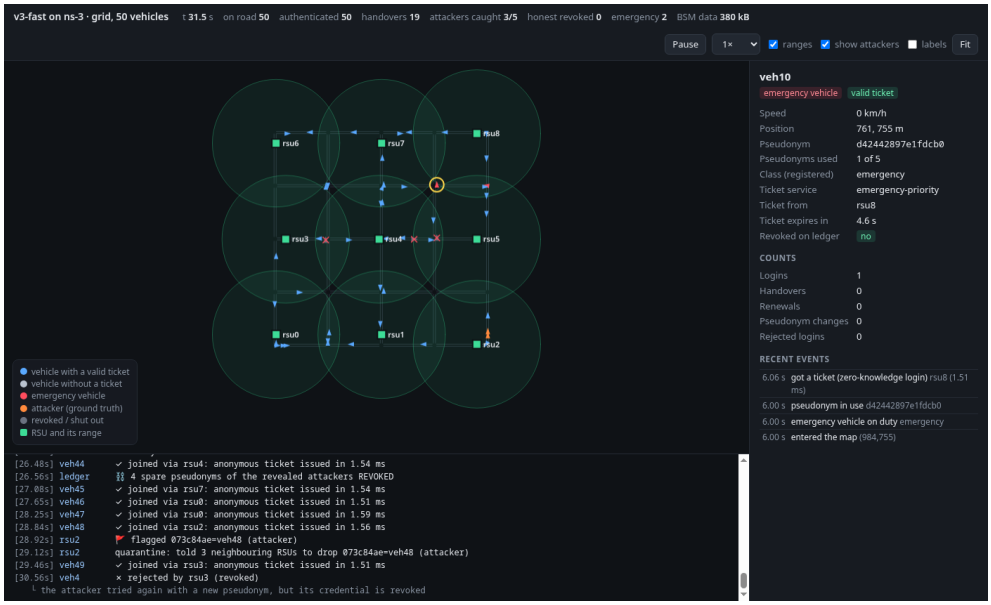


Figure 5.2: Live map of a running ns-3 simulation (grid, 50 vehicles, with emergency vehicles). The selected emergency vehicle shows its registered class and its `emergency-priority` ticket; revoked attackers are marked with a cross; the feed narrates detection and revocation.

5.9 Testing

178 automated tests cover the cryptographic primitives, message encoding, the protocol flows (including replay, forged tickets, swapped certificate requests, class escalation and fake emergency alerts), the detector, the ledger rules and the processor model. The chaincode has its own Go unit tests, and the gateway passes the HTTP conformance suite.

Chapter 6

Results and Discussion

This chapter reports the evaluation. All network results come from ns-3. Each configuration runs with several seeds, which change SUMO's trips, the choice of attackers and the radio's randomness. Tables give the mean and the 95 % confidence interval over the seeds.

6.1 Experimental Setup

Table 6.1: Common parameters of the network experiments

Parameter	Value
Simulated time per run	120 s (180 s for the compromised RSUs, 60 s on LuST)
Scenarios	grid with 50, 100, 150 vehicles; LuST at 06:00, 14:00, 08:00
RSUs	9 (grid, 450 m apart), 300 m nominal range
Seeds	5 (grid comparison), 10 (compromised RSUs), 3 (others)
Attackers	10 % of the vehicles, random position offsets from 30 s
Reports to revoke (K), reveal threshold	2 RSUs; 5 of 9 organizations
Ticket lifetime, pseudonyms per vehicle	30 s; 5
Ledger in sweeps	in-process, 500 ms commit (Section 6.6)
Processors	RSU at measured speed, vehicle 5× slower
Vehicle verification	BSMs of senders within 100 m

Table 6.1 lists the common parameters. *This work* is the scheme of Chapter 4; it is compared with the two baselines of Section 5.7, which run on the same simulator, and in Section 6.9 with published schemes.

Metrics.

- *Ticket*: time from sending a login (or renewal) to holding a valid ticket. *Ticket after 10 s* leaves out the start of a run.

- *Safety traffic*: BSM and TESLA-key bytes per vehicle per second.
- *PDR*: share of broadcasts received by the nodes within 300 m of the sender.
- *CBR*: channel busy ratio measured at the RSUs.
- *V2V accept*: time from sending a BSM to a neighbouring vehicle accepting it.
- *Revoked accepted*: share of BSMs from revoked senders that vehicles still accepted.
- *Caught*: share of attackers reported to the ledger.
- *Honest revoked*: honest vehicles that lost a pseudonym to revocation.
- *Tracked*: share of pseudonym changes the eavesdropper links correctly.

6.2 Comparison on the Grid

Table 6.2: Grid scenario, 150 vehicles: latency and overhead (ns-3, mean ± 95 % CI over 5 seeds)

Variant	Ticket (ms)	Handover (ms)	BSM size (B)	Safety traffic (B/s)	PDR	CBR	V2V accept (ms)
This work	1.59 ± 0.07	1.44 ± 0.09	155 ± 0	301 ± 8	0.995 ± 0.000	0.047 ± 0.001	0.92 ± 0.00
ECDSA-PKI	1.75 ± 0.19	1.52 ± 0.16	117	1170 ± 0	0.973 ± 0.003	0.147 ± 0.004	–
Central-TA	22.49 ± 0.15	22.36 ± 0.21	117	1170 ± 0	0.974 ± 0.003	0.147 ± 0.004	–

Table 6.3: Grid scenario, 150 vehicles: security and privacy

Variant	Revoked accepted	Caught	Honest revoked	Tracked	Report to revocation (s)	OBU load
This work	0.001 ± 0.000	1.00	0.0	0.99 ± 0.01	2.64 ± 0.82	0.0056 ± 0.0003
ECDSA-PKI	–	1.00	0.0	–	2.60 ± 0.82	–
Central-TA	–	1.00	0.0	–	2.45 ± 0.60	–

Caught: share of attackers reported; Revoked accepted: share of BSMs from revoked senders that vehicles still accepted; Tracked: share of pseudonym changes a global eavesdropper links.

Tables 6.2 and 6.3 compare this work with the baselines at 150 vehicles, and Figure 6.1 shows the trends over vehicle density.

Latency. This work issues a ticket in 1.59 ms, the ECDSA-PKI baseline in 1.75 ms; the difference is not significant (Table 6.4), so the zero-knowledge login is as fast as

presenting a certificate, while the RSU learns nothing about which registered vehicle it serves. Centralising the decision (Central-TA) costs 22.49 ms, because every login crosses the backbone and queues at one authority. Handovers take 1.44 ms, again on a par with the certificate baseline. A vehicle accepts a neighbour's BSM after 0.92 ms.

Channel load. This is the largest effect. A certificate-signed BSM at 10 Hz puts 1170 B/s per vehicle on the channel. This work sends 301 B/s, 3.9 times less: most BSMs carry a 16-byte TESLA MAC instead of a signature, and dead-reckoning triggering leaves out the BSMs that neighbours can predict. The channel busy ratio falls from 0.147 to 0.047, and the delivery ratio rises from 0.973 to 0.995 because fewer frames collide.

Revocation. With the revocation filter in RSU beacons, vehicles accepted 0.001 of the BSMs from revoked senders; the baselines model revocation at the RSUs only. Every scheme caught every attacker, and no honest vehicle was revoked.

Privacy. The eavesdropper links 0.99 of the pseudonym changes: changing pseudonyms hides who a vehicle is from the RSU and the ledger, but not its trajectory from an observer who hears every BSM. Mix zones, in which vehicles fall silent while they change pseudonyms, are left to future work (Chapter 7).

Table 6.4: Welch's t-test of this work against each baseline (p-values; bold: $p < 0.05$)

Baseline	Ticket (ms)	Handover (ms)	Safety traffic (B/s)	PDR	CBR
ECDSA-PKI	0.069	0.25	**5.7e-10**	**4.8e-05**	**5.6e-08**
Central-TA	**5.4e-13**	**4e-12**	**5.7e-10**	**2.7e-05**	**4.3e-08**

Table 6.4 tests the differences: a bold entry means that the difference between this work and the baseline is significant at the 5 % level (Welch's t-test over the seeds).

6.3 Real Traffic: LuST

Table 6.6 and Figure 6.2 run this work on real Luxembourg traffic at three times of day, with 60 s runs, since a run of the 08:00 peak with over 800 vehicles takes more than an hour to simulate. Two differences from the grid matter. First, a cut of real traffic starts with hundreds of vehicles already on the road, which all log in during the first seconds. The RSUs' processors keep up (a login costs them well under a

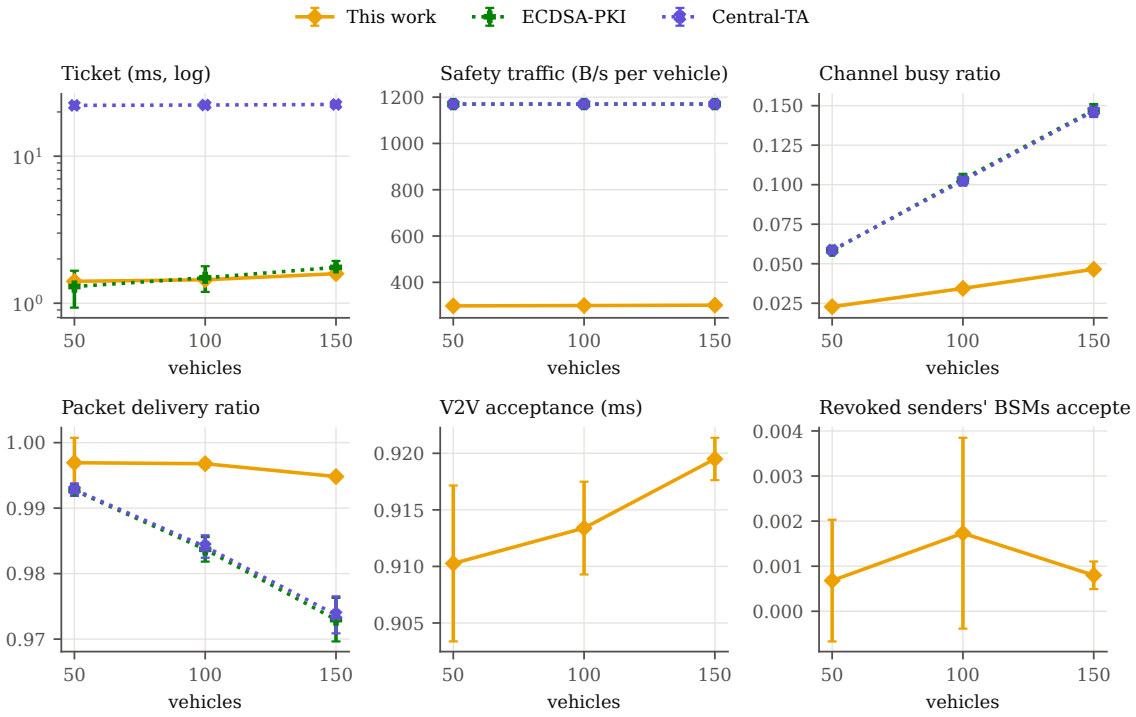


Figure 6.1: Grid scenario over vehicle density (mean and 95 % CI over 5 seeds)

Table 6.5: Grid scenario at 50, 100 and 150 vehicles

Variant	Ticket (ms)	Safety traffic (B/s)	PDR	CBR	V2V accept (ms)
vehicles = 50					
This work	1.41 ± 0.08	298 ± 8	0.997 ± 0.004	0.023 ± 0.001	0.91 ± 0.01
ECDSA-PKI	1.30 ± 0.37	1170 ± 0	0.993 ± 0.001	0.058 ± 0.001	–
Central-TA	22.17 ± 0.17	1170 ± 0	0.993 ± 0.001	0.059 ± 0.002	–
vehicles = 100					
This work	1.44 ± 0.01	300 ± 8	0.997 ± 0.001	0.034 ± 0.001	0.91 ± 0.00
ECDSA-PKI	1.49 ± 0.30	1170 ± 0	0.984 ± 0.002	0.103 ± 0.003	–
Central-TA	22.28 ± 0.21	1170 ± 0	0.984 ± 0.002	0.103 ± 0.003	–
vehicles = 150					
This work	1.59 ± 0.07	301 ± 8	0.995 ± 0.000	0.047 ± 0.001	0.92 ± 0.00
ECDSA-PKI	1.75 ± 0.19	1170 ± 0	0.973 ± 0.003	0.147 ± 0.004	–
Central-TA	22.49 ± 0.15	1170 ± 0	0.974 ± 0.003	0.147 ± 0.004	–

Table 6.6: LuST (Luxembourg) traffic: low (06:00), mid (14:00) and peak (08:00) windows

Variant	Ticket (ms)	Ticket after 10 s (ms)	Safety traffic (B/s)	PDR	CBR	V2V accept (ms)	Caught
real = lust:low							
This work	32.37 ± 6.76	14.54 ± 6.11	392 ± 23	0.994 ± 0.001	0.061 ± 0.004	1.07 ± 0.66	0.89 ± 0.13
real = lust:mid							
This work	41.98 ± 10.49	13.55 ± 10.07	394 ± 22	0.984 ± 0.007	0.127 ± 0.032	5.18 ± 3.12	0.89 ± 0.06
real = lust:peak							
This work	61.55 ± 17.78	22.58 ± 31.91	351 ± 29	0.940 ± 0.015	0.320 ± 0.085	31.72 ± 8.29	0.86 ± 0.08

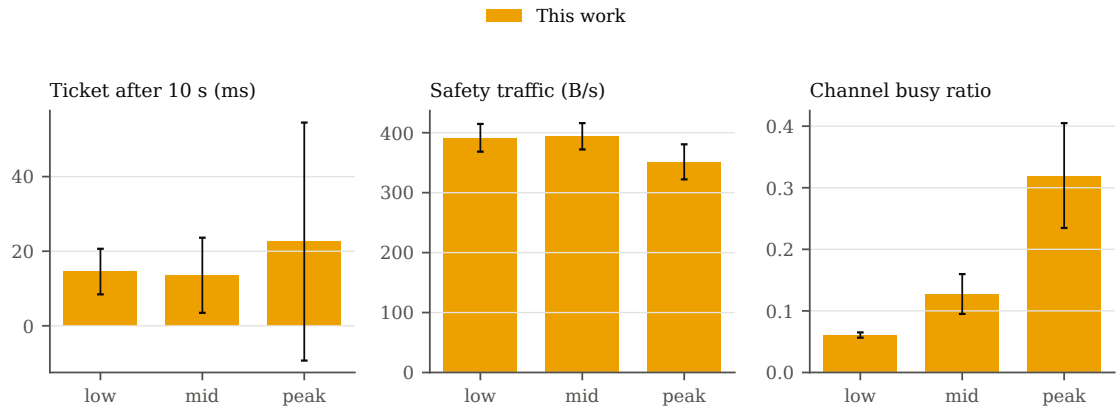


Figure 6.2: LuST traffic at three times of day (mean and 95 % CI over 3 seeds)

millisecond), but the radio does not: hundreds of login requests of about 680 bytes, each with its Merkle path and proof, and the RSUs' ticket responses compete for the channel and wait in the 802.11p queues. In a 15 s run of the 08:00 peak, a login request spent about 110 ms and a ticket response about 240 ms on the air during the first three seconds. The mean ticket time therefore includes this start-up burst. The ticket time after 10 s shows the steady state: in the 08:00 peak, tickets take 22.58 ms once the burst has passed. Second, real roads with many lanes and junctions give more manoeuvres, so dead-reckoning triggering saves less than on the grid: at the peak, this work puts 351 B/s per vehicle on the channel, the channel is busy 0.320 of the time, and 0.940 of the broadcasts arrive.

Two results are weaker than on the grid. At the peak, the channel busy ratio is above the 0.30 at which adaptive signing switches to TESLA, so a neighbour can accept a BSM only once its key is disclosed: V2V acceptance takes 31.72 ms, still well below the 100 ms between two BSMs of a vehicle at 10 Hz. Second, between 85 and 89 % of the attackers are caught on LuST, against all of them on the grid: the attack starts at 30 s of a 60 s run, and an attacker that leaves the coverage of the RSUs before two of them have reported it is not revoked in the run. No honest vehicle was revoked.

6.4 Misbehaviour Detection

Table 6.7: Misbehaviour detection on VeReMi Extension (5 vehicle-disjoint splits)

Detector	F1	FPR	ROC AUC	Attackers reported
Plausibility rules	0.718 ± 0.022	0.0000	–	0.638
Random forest	0.952 ± 0.004	0.0043	0.960	0.867
Gradient boosting	0.952 ± 0.003	0.0027	0.960	0.867
Gradient boosting + history window	0.963 ± 0.003	0.0018	0.969	0.870
+ calibrated threshold	0.963 ± 0.003	0.0018	0.968	0.870

On VeReMi Extension (959,425 messages from 7,494 vehicles, Table 6.7), the plausibility rules alone reach an F1 score of 0.718 with no false positives. They miss subtle attacks such as small constant offsets, which stay within physical limits. Learned models over the same features reach 0.952 (random forest [54]), and gradient boosting with a history window 0.963 at a false-positive rate of 0.0018.

Table 6.8: Detection per attack type

Variant	Caught	Honest revoked	Report to revocation (s)
attack = const_pos			
This work	1.00	0.0	22.70 ± 5.46
attack = const_speed			
This work	1.00	0.0	3.20 ± 3.92
attack = flood			
This work	1.00	0.0	1.62 ± 1.26
attack = random_pos			
This work	1.00	0.0	1.64 ± 1.27
attack = random_speed			
This work	1.00	0.0	1.62 ± 1.07
attack = sudden_stop			
This work	1.00	0.0	3.19 ± 3.75

Table 6.9: Share of attackers

Variant	Caught	Honest revoked	PDR	CBR
attacker_ratio = 0.0				
This work	–	0.0	0.996 ± 0.001	0.034 ± 0.001
attacker_ratio = 0.1				
This work	1.00	0.0	0.997 ± 0.002	0.035 ± 0.002
attacker_ratio = 0.3				
This work	1.00	0.0	0.997 ± 0.001	0.036 ± 0.001

In the simulation, Tables 6.8 and 6.9 show detection per attack type and for an increasing share of attackers. Every attack type is caught, and no honest vehicle is revoked, up to 30 % attackers. The two-report rule matters here: a single RSU's false alarm has no consequence, so honest vehicles keep their pseudonyms even when one RSU misjudges them.

6.5 Compromised RSUs

Table 6.10: Compromised RSUs: 25 % of the RSUs report random honest vehicles (100 vehicles, 10 seeds)

Variant	Caught	Honest revoked	Report to revocation (s)
This work	1.00	3.0 ± 0.8	12.31 ± 4.17

Table 6.10 lets a quarter of the RSUs report random honest pseudonyms every 5 s. Two compromised RSUs together could revoke an honest pseudonym if every report counted the same. With trust-weighted reports (Section 4.9.7), an RSU's weight falls once investigations show its reports were wrong, after which its reports no longer suffice to revoke anyone. Its first false reports still count, so honest vehicles are not fully protected: 3.0 honest pseudonyms are revoked per run on average, while every attacker is still caught.

6.6 Blockchain Performance

Table 6.11 measures the real Fabric 3.1.5 network under a steady load of `RecordAuth` transactions, the most frequent write. With Raft and Fabric's default 500 ms batch timeout, the median commit takes 318 ms, dominated by waiting for the block to be cut. With a 50 ms timeout, Raft commits in 66 ms and SmartBFT in 50 ms at 5 transactions per second. At 20 per second, Byzantine fault tolerance costs more: SmartBFT's median rises to 229 ms against 67 ms for Raft. No transaction failed. Lowering the batch timeout τ from 500 to 50 ms cuts Raft's median from 318 to 66 ms, so the wait for the block to be cut, the term $T_{\text{batch}} \leq \tau$ of Equation (2.7),

Table 6.11: Hyperledger Fabric 3.1.5: commit latency of RecordAuth under load

Ordering service	Offered (tx/s)	Achieved (tx/s)	p50 (ms)	p95 (ms)
Raft, 500 ms batch timeout	5	5.0	318	522
	10	10.0	317	520
	20	20.0	258	471
Raft, 50 ms batch timeout	5	5.0	66	69
	10	10.0	66	69
	20	20.0	67	178
SmartBFT, 50 ms batch timeout	5	5.0	50	73
	10	10.0	51	74
	20	20.0	229	515

dominates commit latency. These latencies confirm the design principle: a commit takes tens to hundreds of milliseconds, too slow for the path of a safety message, but fast enough for authorization records and revocations, which the RSUs bridge with PREAUTH notes and quarantine.

6.7 Sensitivity to the Vehicle Processor

Table 6.12: Vehicle processor speed: measured times × 1, 5, 10

Variant	Ticket (ms)	V2V accept (ms)	OBU load	V2V verified
obu_scale = 1				
This work	1.59 ± 0.17	0.55 ± 0.00	0.0011 ± 0.0001	0.995 ± 0.002
obu_scale = 5				
This work	1.59 ± 0.17	0.92 ± 0.01	0.0056 ± 0.0006	0.995 ± 0.002
obu_scale = 10				
This work	1.59 ± 0.17	1.39 ± 0.00	0.0113 ± 0.0011	0.995 ± 0.002

The speed of real on-board units varies widely: software implementations on embedded processors verify tens of signatures per second, while units with a cryptographic accelerator verify thousands. Instead of relying on a single assumed factor, Table 6.12 repeats the comparison with vehicle processors as fast as the development PC, 5 times slower, and 10 times slower.

Table 6.13: Emergency vehicles (2 % of the honest vehicles) on LuST traffic: ticket times

Variant	EV ticket (ms)	EV ticket p95 (ms)	Other ticket (ms)	Other ticket p95 (ms)
real = lust:mid				
EV, no priority	40.31 ± 19.85	161.8 ± 136.4	42.45 ± 8.58	170.8 ± 38.9
EV + priority	12.06 ± 4.08	30.1 ± 21.2	40.17 ± 5.98	172.4 ± 21.8
EV + priority, fake-EV attack	11.87 ± 6.34	30.1 ± 21.2	39.74 ± 5.77	172.0 ± 22.1

Table 6.14: Emergency vehicles on LuST traffic: warnings and the fake-emergency attack

Variant	Warned in 1 s	Alert delay (ms)	Fake alerts accepted	Fake logins denied	Caught
real = lust:mid					
EV, no priority	0.988 ± 0.006	1.25 ± 0.01	0	0.0	0.89 ± 0.06
EV + priority	0.980 ± 0.043	1.27 ± 0.03	0	0.0	0.89 ± 0.06
EV + priority, fake-EV attack	0.985 ± 0.014	1.27 ± 0.03	0	24.0 ± 4.3	0.88 ± 0.09

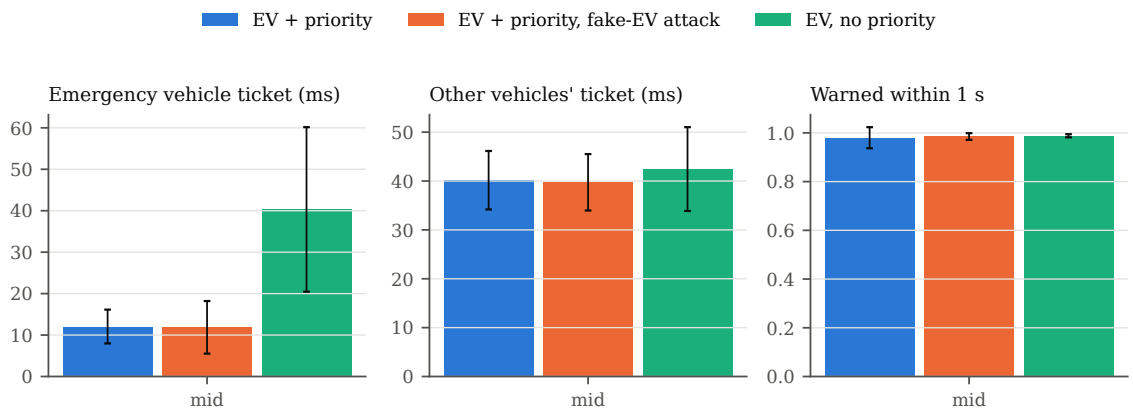


Figure 6.3: Emergency vehicles on LuST: ticket times and warning coverage

6.8 Emergency Vehicles

Tables 6.13 and 6.14 evaluate the extension of Section 4.10 on the 14:00 LuST traffic (60 s runs), where the start-up burst creates long radio queues at the RSUs. The first compares the ticket time of emergency vehicles with that of all other vehicles, with and without the priority lane. The second reports the share of vehicles within 300 m of an emergency vehicle that held a verified alert from it at most 1 s old, the alert delay, and the outcome of the fake-emergency attack. The design requirement is that no fake alert is ever accepted, and that every fake emergency login is denied by the policy or rejected because the pseudonym is already quarantined.

6.9 Comparison with Published Schemes

Published schemes report their results on different simulators, hardware and libraries, so their numbers cannot be placed next to ours directly: BPA [11], for example, measured an elliptic-curve point multiplication at 0.233 ms, our implementation at 0.016 ms. The usual fair comparison counts the cryptographic operations of one authentication and prices them with one set of unit costs. Table 6.15 does this with BPA's published unit costs, the only set that covers all the schemes. For BPA, ZAMA [9] and B-TSCA [12] it copies the operation counts, times and message sizes that BPA publishes (its Tables 4 and 5). For our scheme it counts the operations of the protocol in Chapter 4 and prices only the point multiplications and hashes, which dominate; the message sizes are measured in the ns-3 runs.

The comparison is mixed, and Table 6.15 shows why. In computation our login is on a par with BPA, which also builds on elliptic-curve operations (eight point multiplications each), cheaper than ZAMA, whose zero-knowledge proof uses modular exponentiations, and about five times cheaper than B-TSCA, whose two pairings alone cost 9.5 ms. Our messages are larger: the Merkle path (nine hashes, 288 B) travels with every proof, which is what lets the RSU check membership without learning which registered vehicle it is talking to. Our handover costs more than BPA's re-authentication, because the new RSU checks a signed ticket and a signed request instead of looking up a shared trust key; RSU clusters (Section 4.9) avoid most of these handovers instead.

Table 6.15: Cost of one authentication and one re-authentication (handover), priced with BPA's unit costs (point multiplication 0.233 ms, hash 0.0055 ms, pairing 4.756 ms)

Scheme	Ledger on the	Authentication		Re-authentication		Messages (B)	
	login path	operations	ms	operations	ms	auth.	re-auth.
ZAMA [9]	no	$11E + 5M + \ldots$	2.64	$2E + \ldots$	1.76	1200	472
B-TSCA [12]	lookup	$6E + 2P + 4H + \ldots$	10.11	$6E + 2P + 2H + \ldots$	10.10	72	136
BPA [11]	re-auth.	$8PM + 3H + \ldots$	1.91	$7PM + H + \ldots$	1.65	386	240
This work	no	$8PM + 15H$	1.95	$10PM + 6H$	2.36	716	410

PM: point multiplication, H: hash, E: modular exponentiation, M: modular multiplication, P: bilinear pairing; "$\ldots$" are the cheaper operations listed in [11]. Rows 1–3 and their times are as published in [11] (its Table 5 gives 386 B for BPA's authentication, its Figure 8 368 B). This work: authentication = vehicle checks the beacon (2 PM) and proves (1 PM), RSU checks the Merkle path (10 H) and the proof (2 PM) and signs the ticket (1 PM), vehicle checks the ticket (2 PM); handover = vehicle checks the beacon and signs the request (3 PM), the new RSU checks the old ticket and the request (4 PM) and signs a new ticket (1 PM), vehicle checks it (2 PM). Message sizes: request plus ticket response, measured in ns-3.

The largest difference is architectural. BAAIoV [10] authenticates every vehicle through a Hyperledger Fabric transaction and reports 150 ms at 50 vehicles rising to 187 ms at 150 vehicles, measured with SUMO and Fabric but without a radio model. Our design keeps the ledger off the login path: on the grid with 150 vehicles a login completes in 1.59 ms end to end over 802.11p, while the ledger records it in the background in the tens to hundreds of milliseconds that Table 6.11 measures. The two blockchain-and-ZKP schemes closest to ours, Wang et al. [40] and the emergency-vehicle scheme of Li et al. [41], are compared in Chapter 3 by their properties only.

6.10 Discussion

The research question of the Phase I review asked whether ZKP authentication combined with permissioned-blockchain authorization can work without unacceptable latency, computation or communication overhead. On the measured network the answer is yes, under two conditions. First, the ledger must stay off the safety-message path. Proofs are verified at the RSU in a fraction of a millisecond, while ledger commits take tens to hundreds of milliseconds and are bridged by PREAUTH and quarantine. Second, per-message authentication must be cheap: with TESLA and dead-reckoning triggering this work puts 3.9 times less safety traffic on the channel than certificate-signed BSMs, and its login is as fast as presenting a certificate, without giving the RSU or the ledger the vehicle's identity.

Threats to validity.

- Radio and traffic are simulated. ns-3 and SUMO are the accepted tools for this purpose, and LuST adds real traffic, but no field test was done.
- Processing costs were measured on a PC and scaled for vehicles. The processor sensitivity sweep bounds the effect.
- The parameter sweeps use an in-process ledger with the measured Raft commit time, and the real Fabric network was measured separately on one machine. A wide-area deployment adds network latency to commits.
- The investigation that drives trust weights is simulated with ground truth.
- Attackers follow the VeReMi attack types and a fake-emergency attack. Adaptive attackers who stay just inside the plausibility limits are harder to detect, as the VeReMi results for small offsets show.

Chapter 7

Conclusion and Future Work

7.1 Summary

This project designed, implemented and evaluated privacy-preserving authentication and authorization for VANETs that combines zero-knowledge proofs with a permissioned blockchain:

- Vehicles prove registration and class with a Schnorr proof against a Merkle root on the ledger, bound to the RSU, a fresh nonce and the requested service.
- RSUs decide access with ledger policies and issue short-lived tickets.
- Misbehaving pseudonyms are reported to the ledger and revoked by two reports.
- A threshold of RSU organizations can unmask a revoked vehicle.
- The ledger is kept off the safety-message path.

The complete system runs on ns-3 with SUMO mobility on a grid and on real Luxembourg traffic, with measured cryptographic costs, against certificate-based and centralised baselines and published schemes. Its login is as fast as presenting a certificate and costs about as much computation as BPA, and it puts 3.9 times less safety traffic on the channel than certificate-signed messages. Its revocation filter lets vehicles, not only RSUs, ignore revoked senders. On VeReMi Extension, the detector improves from an F1 score of 0.718 to 0.963. Emergency vehicles are supported as a policy-restricted class whose alerts cannot be forged under a normal ticket.

7.2 Limitations

- The evaluation is simulation-based; vehicle processing speed is an assumption bounded by a sensitivity sweep, not a measurement on OBU hardware.
- The TA knows which pseudonyms belong to which vehicle.
- A global eavesdropper links most pseudonym changes by position and speed.
- Login and handover messages are larger than in BPA, because the Merkle path travels with every proof.
- A coalition of t RSU organizations can unmask any vehicle; this is the intended accountability, but it requires that fewer than t organizations collude.
- Learned detectors did not transfer reliably between road networks; the scheme uses rules and the signal-strength check.
- The processor part of the emergency priority lane is modelled approximately: work already queued keeps its finish times. Any radio can use a higher access category, so the radio part relies on the RSU's cap on priority answers rather than on enforcement.

7.3 Future Work

1. Remove the remaining privacy gaps: anonymous credentials such as BBS [55], so that the TA no longer knows the pseudonyms, and cooperative mix zones [42, 43] against a global eavesdropper.
2. Measure the cryptographic operations on OBU-class hardware, for example a Raspberry Pi or a commercial V2X unit, to replace the processor scaling factor.
3. Run the full parameter sweeps against the real Fabric network, distributed over several machines.
4. Verify the authentication and revocation protocols formally with Tamarin or ProVerif.
5. Evaluate stronger attackers: Sybil attacks with several own pseudonyms, adaptive attackers that stay within plausibility limits, colluding vehicles, and flood-

ing of RSUs.

6. Train detectors that transfer between cities, using data from several maps or sequence models, and let vehicles vote cooperatively.
7. Add C-V2X (LTE/5G sidelink) next to 802.11p, for example with ns-3's 5G-LENA module.
8. Measure post-quantum signatures, such as Falcon or ML-DSA, whose size TESLA largely hides.
9. Encode messages in the IEEE 1609.2 / ETSI TS 103 097 formats, and compare with SCMS.
10. Scale to a full day of LuST and to further cities.
11. Extend the emergency extension with traffic-signal preemption through live SUMO control.

7.4 Closing Remarks

Authentication in VANETs is not one problem but a balance between trust, privacy and the capacity of a shared radio channel. Our measurements show that the balance can be struck. Zero-knowledge proofs keep identities private, the permissioned ledger keeps authorization and revocation shared and auditable, and the cost stays within what safety applications can afford. This holds when the ledger is used for state that must be shared and audited, not for every message.

Appendix A

Reproducing the Results

The repository's `README.md` and `docs/how-it-works.md` describe setup in detail. On Linux, `./setup.sh` creates a Python environment with ns-3 and SUMO. The main commands are:

Command	Result
`make test`	all automated tests
`make live`	one ns-3 run narrated in the terminal
`make viz`	the same run as a live map in the browser
`make scenario-3obu`, `-low`, `-veryhigh`	demonstration scenarios: one normal vehicle, one attacker and one emergency vehicle; 30 vehicles; 300 vehicles
`make evaluate-all`	the sweeps behind Chapter 6 (about a day on 8 cores)
`make emergency`	the emergency-vehicle sweep
`make detector-benchmark`	the VeReMi Extension benchmark
`make ledger-bench`	the Fabric Raft and SmartBFT benchmark
`make fabric-up`, `make gateway`	a real Fabric network and its REST gateway

Every table, figure and number in Chapters 6 and in the abstract is generated from the experiment outputs by `python experiments/thesis_data.py`, which writes the folder `docs/report/generated`. Each run folder contains `config.json` with every option of the run, so any result can be repeated exactly. Every run also has its `metrics.csv` with one row per event, from which all metrics are computed.

Bibliography

[1] SAE International, "SAE J2735: V2X communications message set dictionary," 2020.

[2] ETSI, "ETSI EN 302 637-2 V1.4.1: Intelligent transport systems; vehicular communications; basic set of applications; part 2: Specification of cooperative awareness basic service," 2019.

[3] D. Jiang and L. Delgrossi, "IEEE 802.11p: Towards an international standard for wireless access in vehicular environments," in *Proc. IEEE Vehicular Technology Conference (VTC Spring)*, 2008, pp. 2036–2040.

[4] J. B. Kenney, "Dedicated short-range communications (DSRC) standards in the United States," *Proceedings of the IEEE*, vol. 99, no. 7, pp. 1162–1182, 2011.

[5] M. Gruteser and B. Hoh, "On the anonymity of periodic location samples," in *Proc. Security in Pervasive Computing (SPC)*, ser. LNCS, vol. 3450. Springer, 2005, pp. 179–192.

[6] M. Raya and J.-P. Hubaux, "Securing vehicular ad hoc networks," *Journal of Computer Security*, vol. 15, no. 1, pp. 39–68, 2007.

[7] B. Brecht, D. Therriault, A. Weimerskirch, W. Whyte, V. Kumar, T. Hehn, and R. Goudy, "A security credential management system for V2X communications," *IEEE Transactions on Intelligent Transportation Systems*, vol. 19, no. 12, pp. 3850–3871, 2018.

[8] A. R. Rasheed, R. N. Mahapatra, and F. G. Hamza-Lup, "Adaptive group-based zero knowledge proof-authentication protocol in vehicular ad hoc networks," *IEEE Transactions on Intelligent Transportation Systems*, vol. 21, no. 2, pp. 867–881, 2020.

[9] N. Xi, W. Li, L. Jing, and J. Ma, "ZAMA: A ZKP-based anonymous mutual authentication scheme for the IoV," *IEEE Internet of Things Journal*, vol. 9, no. 22, pp. 22 903–22 913, 2022.

[10] A. Alsaeed, S. Almowuena, and A. N. Alyahya, "BAAIoV: A blockchain-based authentication and authorization framework for secure and reliable Internet of Vehicles communication," *IEEE Access*, 2025.

[11] J. Li, Y. Lin, Y. Li, Y. Zhuang, and Y. Cao, "BPA: A novel blockchain-based privacy-preserving authentication scheme for the Internet of Vehicles," *Electronics*, vol. 13, no. 10, p. 1901, 2024.

[12] C. Wang, J. Shen, J. Lai, and J. Liu, "B-TSCA: Blockchain assisted trustworthiness scalable computation for V2I authentication in VANETs," *IEEE Transactions on Emerging Topics in Computing*, vol. 9, no. 3, pp. 1386–1396, 2021.

[13] Certicom Research, "SEC 2: Recommended elliptic curve domain parameters," Standards for Efficient Cryptography, version 2.0, 2010.

[14] National Institute of Standards and Technology, "Secure hash standard (SHS)," NIST, FIPS PUB 180-4, 2015.

[15] C. P. Schnorr, "Efficient signature generation by smart cards," *Journal of Cryptology*, vol. 4, no. 3, pp. 161–174, 1991.

[16] A. Fiat and A. Shamir, "How to prove yourself: Practical solutions to identification and signature problems," in *Advances in Cryptology – CRYPTO '86*, ser. LNCS, vol. 263. Springer, 1987, pp. 186–194.

[17] R. C. Merkle, "A digital signature based on a conventional encryption function," in *Advances in Cryptology – CRYPTO '87*, ser. LNCS, vol. 293. Springer, 1988, pp. 369–378.

[18] A. Shamir, "How to share a secret," *Communications of the ACM*, vol. 22, no. 11, pp. 612–613, 1979.

[19] P. Wuille, J. Nick, and T. Ruffing, "BIP 340: Schnorr signatures for secp256k1," Bitcoin Improvement Proposal, 2020.

[20] A. Perrig, R. Canetti, J. D. Tygar, and D. Song, "Efficient authentication and signing of multicast streams over lossy channels," in *Proc. IEEE Symposium on Security and Privacy*, 2000, pp. 56–73.

[21] L. Lamport, "Password authentication with insecure communication," *Communications of the ACM*, vol. 24, no. 11, pp. 770–772, 1981.

[22] Certicom Research, "SEC 4: Elliptic curve Qu-Vanstone implicit certificate scheme (ECQV), version 1.0," Standards for Efficient Cryptography Group, Tech. Rep., 2013.

[23] B. H. Bloom, "Space/time trade-offs in hash coding with allowable errors," *Communications of the ACM*, vol. 13, no. 7, pp. 422–426, 1970.

[24] E. Androulaki, A. Barger, V. Bortnikov *et al.*, "Hyperledger Fabric: A distributed operating system for permissioned blockchains," in *Proc. Thirteenth EuroSys Conference*, 2018, pp. 30:1–30:15.

[25] D. Ongaro and J. Ousterhout, "In search of an understandable consensus algorithm," in *Proc. USENIX Annual Technical Conference*, 2014, pp. 305–319.

[26] A. Barger, Y. Manevich, H. Meir, and Y. Tock, "A Byzantine fault-tolerant consensus library for Hyperledger Fabric," in *Proc. IEEE International Conference on Blockchain and Cryptocurrency (ICBC)*, 2021, pp. 1–9.

[27] L. Lamport, R. Shostak, and M. Pease, "The Byzantine generals problem," *ACM Transactions on Programming Languages and Systems*, vol. 4, no. 3, pp. 382–401, 1982.

[28] M. Castro and B. Liskov, "Practical Byzantine fault tolerance," in *Proc. 3rd USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 1999, pp. 173–186.

[29] R. W. van der Heijden, T. Lukaseder, and F. Kargl, "VeReMi: A dataset for comparable evaluation of misbehavior detection in VANETs," in *Proc. SecureComm*, ser. LNICST, vol. 254. Springer, 2018, pp. 318–337.

[30] J. Kamel, M. Wolf, R. W. van der Heijden, A. Kaiser, P. Urien, and F. Kargl, "VeReMi extension: A dataset for comparable evaluation of misbehavior detection in VANETs," in *Proc. IEEE International Conference on Communications (ICC)*, 2020, pp. 1–6.

[31] IEEE, "IEEE standard for wireless access in vehicular environments – security services for application and management messages (IEEE std 1609.2-2022)," 2023.

[32] H. Zhong, S. Han, J. Cui, J. Zhang, and Y. Xu, "Privacy-preserving authentication scheme with full aggregation in VANET," *Information Sciences*, vol. 476, pp. 211–221, 2019.

[33] Z. Liu, N. Yao, S. Bai *et al.*, "A cooperative ECC-based authentication protocol for VANETs," *Scientific Reports*, vol. 15, p. 40837, 2025.

[34] J. Qi, T. Gao, X. Deng, and C. Zhao, "A pseudonym-based certificateless privacy-preserving authentication scheme for VANETs," *Vehicular Communications*, vol. 38, p. 100535, 2022.

[35] L. Li, H. Deng, Z. Zhai, and S.-L. Peng, "Privacy-preserving authentication based on PUF for VANETs," *Future Internet*, vol. 16, no. 9, p. 326, 2024.

[36] M. Gupta, R. Kumar, S. Shekhar, B. Sharma *et al.*, "Game theory-based authentication framework to secure Internet of Vehicles with blockchain," *Sensors*, vol. 22, no. 14, p. 5119, 2022.

[37] Q. Xie, Z. Ding, W. Tang, D. He, and X. Tan, "Provable secure and lightweight blockchain-based V2I handover authentication and V2V broadcast protocol for VANETs," *IEEE Transactions on Vehicular Technology*, vol. 72, no. 12, pp. 15 200–15 212, 2023.

[38] I. M. Varma and N. Kumar, "ZKP-based lightweight authentication protocol during handovers in vehicular networks," in *Proc. IEEE Global Communications Conference (GLOBECOM)*, 2023, pp. 868–873.

[39] W. Jiang and Z. Guo, "An anonymous authentication scheme for Internet of Vehicles based on TRUG-PBFT master-slave chains and zero-knowledge proof," *IEEE Internet of Things Journal*, 2024.

[40] S. Wang, Q. Huang, R. Yan, J. Ma, and X. Xie, "A privacy-enhanced authentication scheme for VANETs based on blockchain and zero-knowledge proof," *Vehicular Communications*, vol. 56, p. 100976, 2025.

[41] H. Li, D. Oza, M. Yoshida, and T. Noguchi, "Privacy-preserving emergency vehicle authentication scheme using zero-knowledge proofs and blockchain," *IoT*, vol. 7, no. 2, p. 35, 2026.

[42] A. R. Beresford and F. Stajano, "Location privacy in pervasive computing," *IEEE Pervasive Computing*, vol. 2, no. 1, pp. 46–55, 2003.

[43] J. Freudiger, M. Raya, M. Félegyházi, P. Papadimitratos, and J.-P. Hubaux, "Mix-zones for location privacy in vehicular networks," in *Proc. ACM Workshop on Wireless Networking for Intelligent Transportation Systems (WiN-ITS)*, 2007.

[44] M. Bellare, J. A. Garay, and T. Rabin, "Fast batch verification for modular exponentiation and digital signatures," in *Advances in Cryptology – EUROCRYPT '98*, ser. LNCS, vol. 1403. Springer, 1998, pp. 236–250.

[45] A. Jøsang and R. Ismail, "The beta reputation system," in *Proc. 15th Bled Electronic Commerce Conference*, 2002.

[46] P. A. Lopez, M. Behrisch, L. Bieker-Walz, J. Erdmann, Y.-P. Flötteröd, R. Hilbrich, L. Lücken, J. Rummel, P. Wagner, and E. Wießner, "Microscopic traffic simulation using SUMO," in *Proc. 21st IEEE International Conference on Intelligent Transportation Systems (ITSC)*, 2018, pp. 2575–2582.

[47] L. Codecá, R. Frank, and T. Engel, "Luxembourg SUMO traffic (LuST) scenario: 24 hours of mobility for vehicular networking research," in *Proc. IEEE Vehicular Networking Conference (VNC)*, 2015, pp. 1–8.

[48] G. F. Riley and T. R. Henderson, "The ns-3 network simulator," in *Modeling and Tools for Network Simulation*. Springer, 2010, pp. 15–34.

[49] M. Dworkin, "Recommendation for block cipher modes of operation: Galois/Counter Mode (GCM) and GMAC," NIST, Special Publication 800-38D, 2007.

[50] J. H. Friedman, "Greedy function approximation: A gradient boosting machine," *Annals of Statistics*, vol. 29, no. 5, pp. 1189–1232, 2001.

[51] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion *et al.*, "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[52] B. L. Welch, "The generalization of 'Student's' problem when several different population variances are involved," *Biometrika*, vol. 34, no. 1/2, pp. 28–35, 1947.

[53] D. Johnson, A. Menezes, and S. Vanstone, "The elliptic curve digital signature algorithm (ECDSA)," *International Journal of Information Security*, vol. 1, no. 1, pp. 36–63, 2001.

[54] L. Breiman, "Random forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.

[55] D. Boneh, X. Boyen, and H. Shacham, "Short group signatures," in *Advances in Cryptology – CRYPTO 2004*, ser. LNCS, vol. 3152. Springer, 2004, pp. 41–55.